Welcome to Machine Learning 2026

AARHUS UNIVERSITY

The Hand-ins

- 3 Mandatory Projects
  - Must be **approved** to attend exam.

- To be done in groups of 3.
  - Need permission from TA to make smaller groups.
  - Need a good reason.

- Use of GenAI:
  - We discourage using GenAI for doing your projects.
  - No grading, so only your own learning that is harmed by using it.



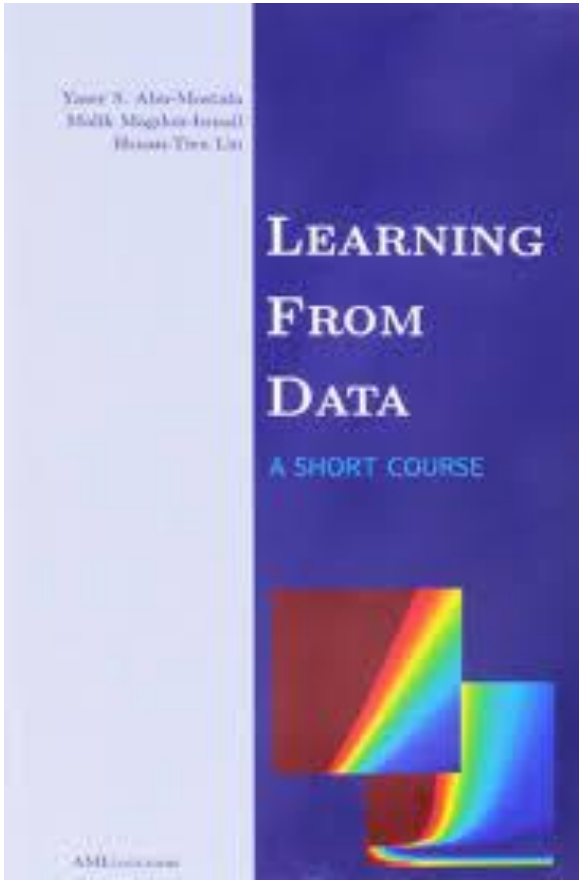
The Exam

- Multiple Choice
  - 2 hours.
  - No aids (so we can ask easier questions).
    - More details on computers towards end of semester.

- Old exam sets can be found on Brightspace.
  - Some older sets contain questions about Hidden Markov Models
    - Is not curriculum anymore

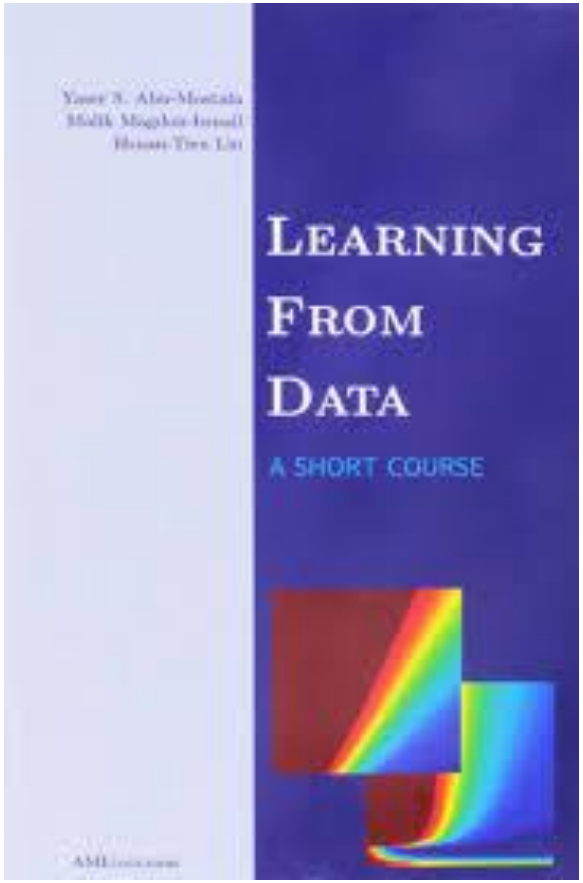


The Material



- Most ML books: Start with >100 pages of probability theory before any ML algorithm or problem is studied.
- We picked what we think is the best material for each topic.
- Downside: Minor inconsistencies in terminology.

The Material



- Other material can be found as pdf on Brightspace page.
- Buy the book at Stakbogladen.

The Study Cafe

CS Studiecafe



Starting September 2nd

Weekday	Time
Mondays	9-10
Wednesdays	11-12
Fridays	9-10

https://studerende.au.dk/studier/fagportaler/datalogi/studiemiljoe/cs-studiecafe

The TA Sessions

- 3 hours per week.
- **At least** look through exercises before coming.
- **Preferably** try to solve them before coming.
- **Do** install any Python packages etc. before coming.

- Mix of coding and theoretical exercises.

- Curriculum includes hand-ins and exercises.
- Solving exercises prepare you for the exam.
  - And teaches you to do ML in practice.



Welcome to Machine Learning 2026

AARHUS UNIVERSITY

Machine Learning

What is Machine Learning?

Machine Learning



- First Questions:
  - What is Machine Learning?

- Learning from data…
  - What does this actually mean?

Machine Learning

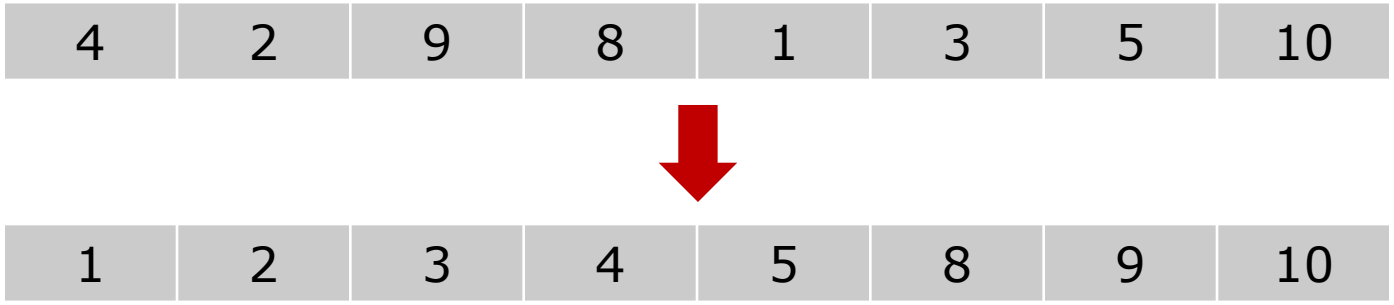
- Learning from data…
  - Said shortly:
    - Algorithms that produce algorithms based on data.
      - Why?
      - How?
    - Let's start with *why*.

Machine Learning

- Classic Algorithm Design
  - Ingredients:
    - Clearly specified problem: Input and desired output.
    - Programmer/Researcher/Student thinks hard about it and designs an algorithm.
    - Algorithm is always correct, can analyse its performance theoretically.

Machine Learning

- Classic Algorithm Design
  - Example:
  - Input: Array of n numbers.
  - Output: Array with the numbers sorted.



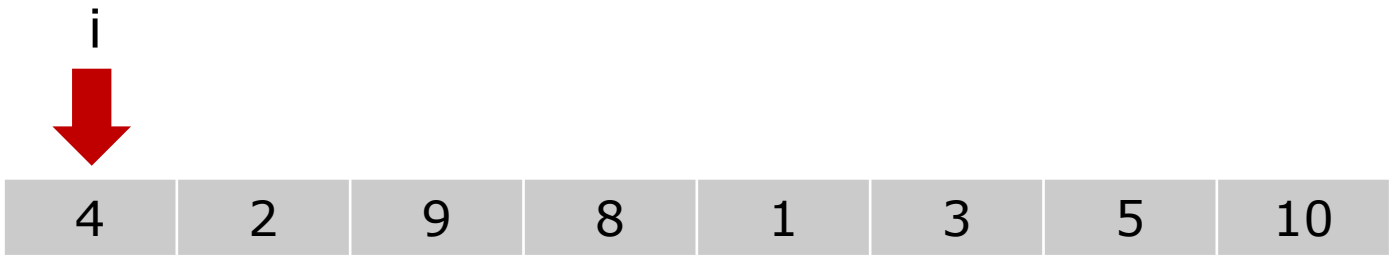
Machine Learning

- Classic Algorithm Design
  - Example:
  - Input: Array of n numbers.
  - Output: Array with the numbers sorted.

4	2	9	8	1	3	5	10

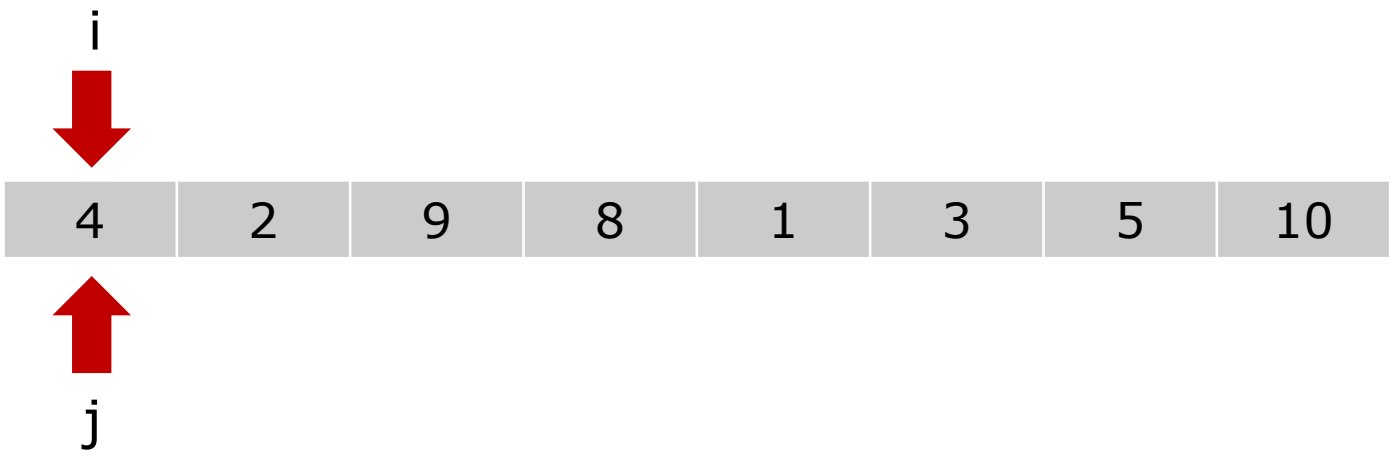
- Algorithm:
  - For i=0,…,n-1: //From left to right
    - j=i //Put elm. j to cor. pos.
    - While j>0 AND A[j]<A[j-1]: //Swap while smaller
      - Swap(A[j],A[j-1])
      - j = j-1

Machine Learning



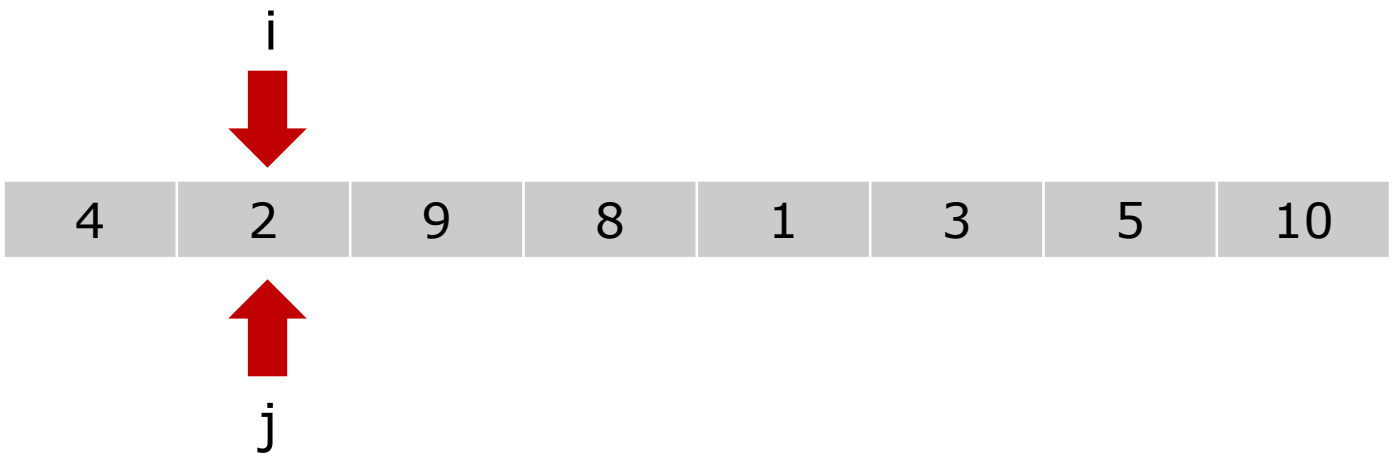
- Algorithm:
  - For i=0,…,n-1:
    - j=i
    - While j>0 AND A[j]<A[j-1]:
      - Swap(A[j],A[j-1])
      - j = j-1

Machine Learning



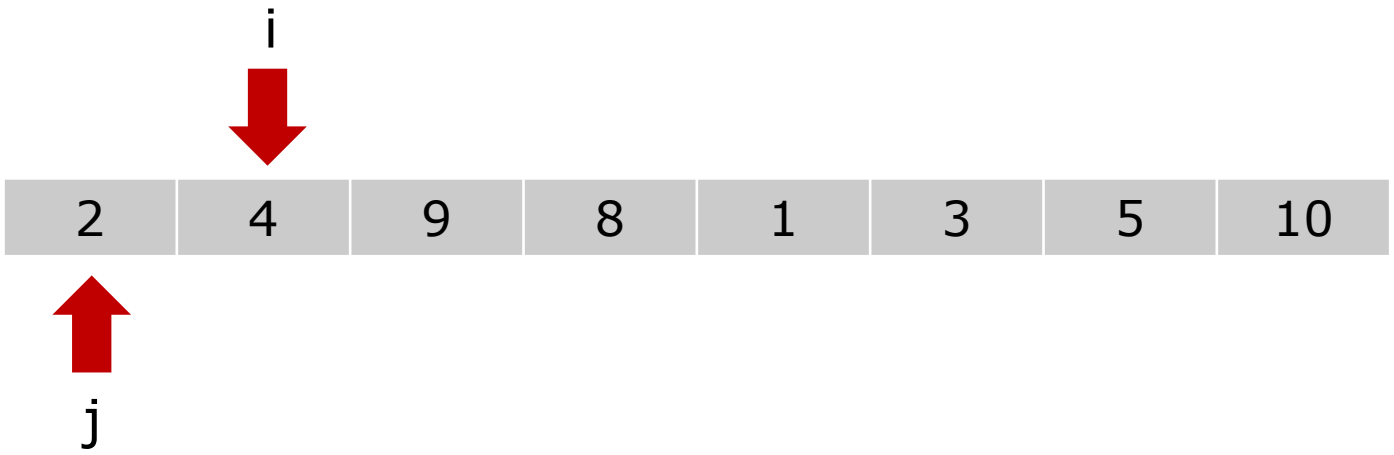
- Algorithm:
  - For i=0,…,n-1:
    - j=i
    - While j>0 AND A[j]<A[j-1]:
      - Swap(A[j],A[j-1])
      - j = j-1

Machine Learning



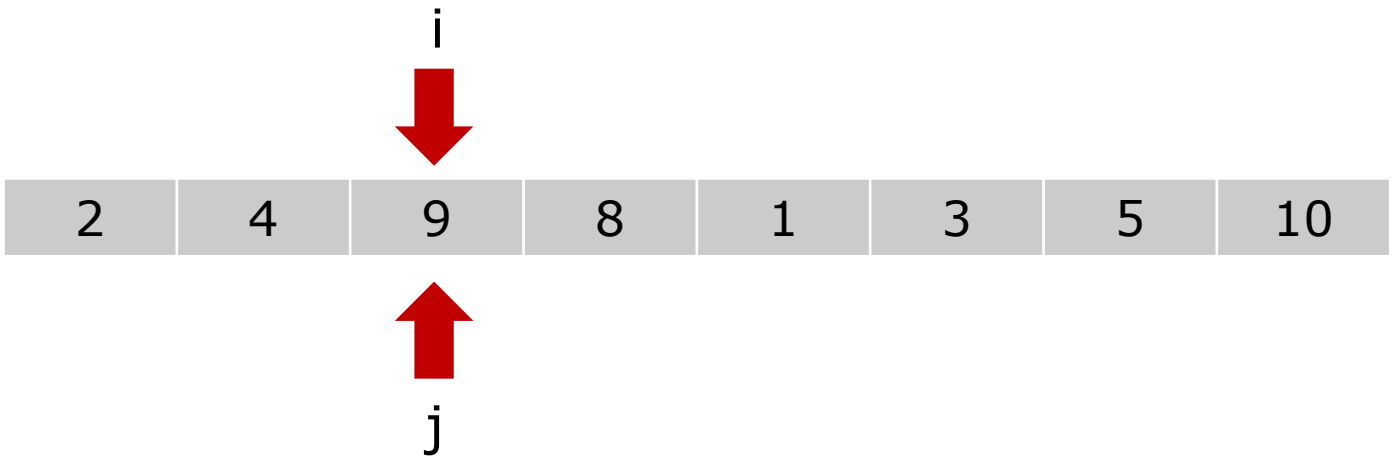
- Algorithm:
  - For i=0,…,n-1:
    - j=i
    - While j>0 AND A[j]<A[j-1]:
      - Swap(A[j],A[j-1])
      - j = j-1

Machine Learning



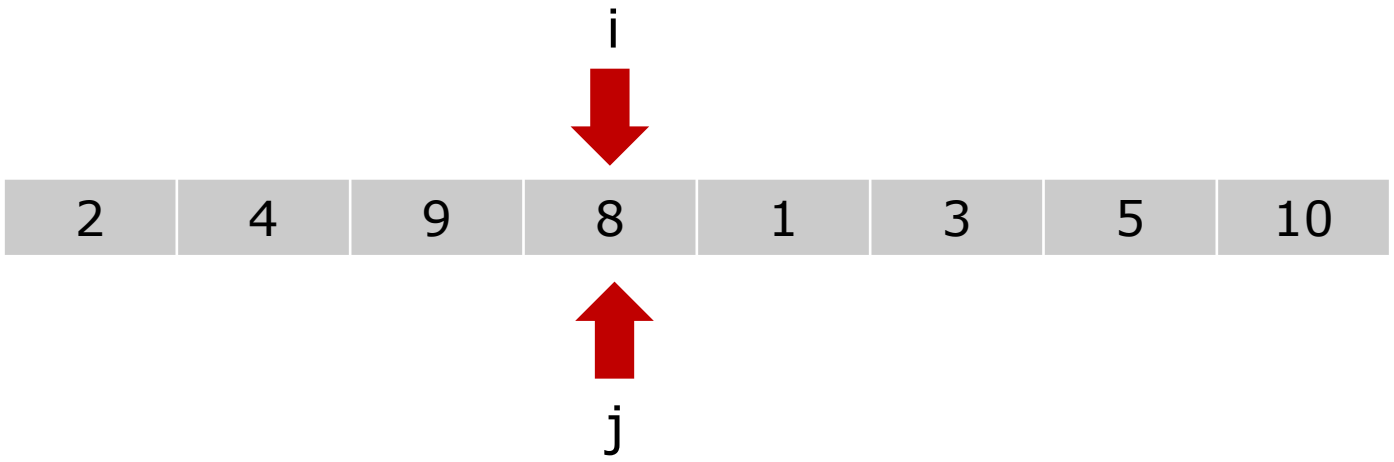
- Algorithm:
  - For i=0,…,n-1:
    - j=i
    - While j>0 AND A[j]<A[j-1]:
      - Swap(A[j],A[j-1])
      - j = j-1

Machine Learning



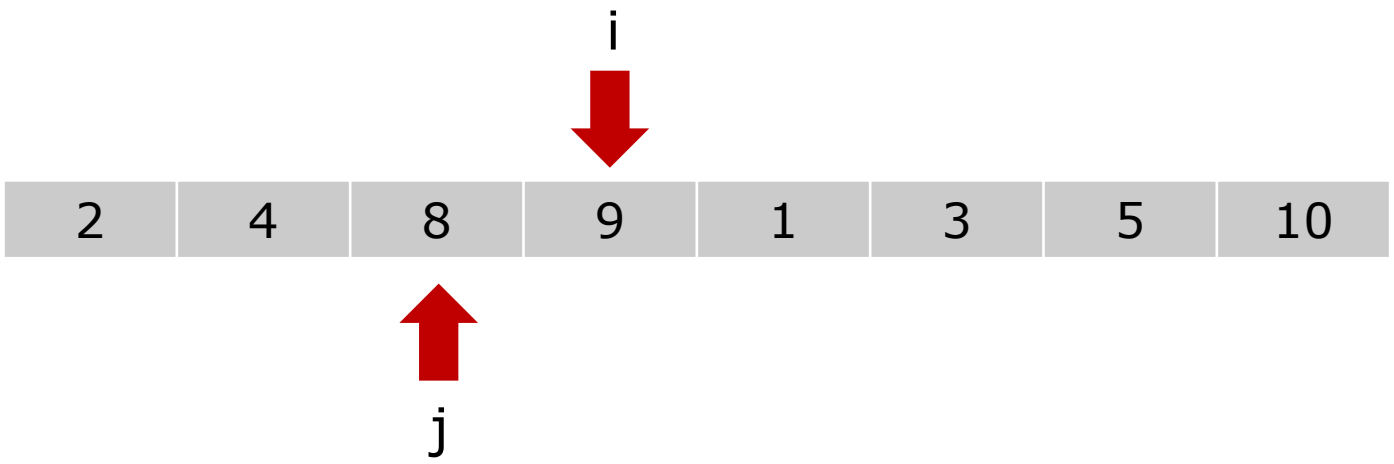
- Algorithm:
  - For i=0,…,n-1:
    - j=i
    - While j>0 AND A[j]<A[j-1]:
      - Swap(A[j],A[j-1])
      - j = j-1

Machine Learning



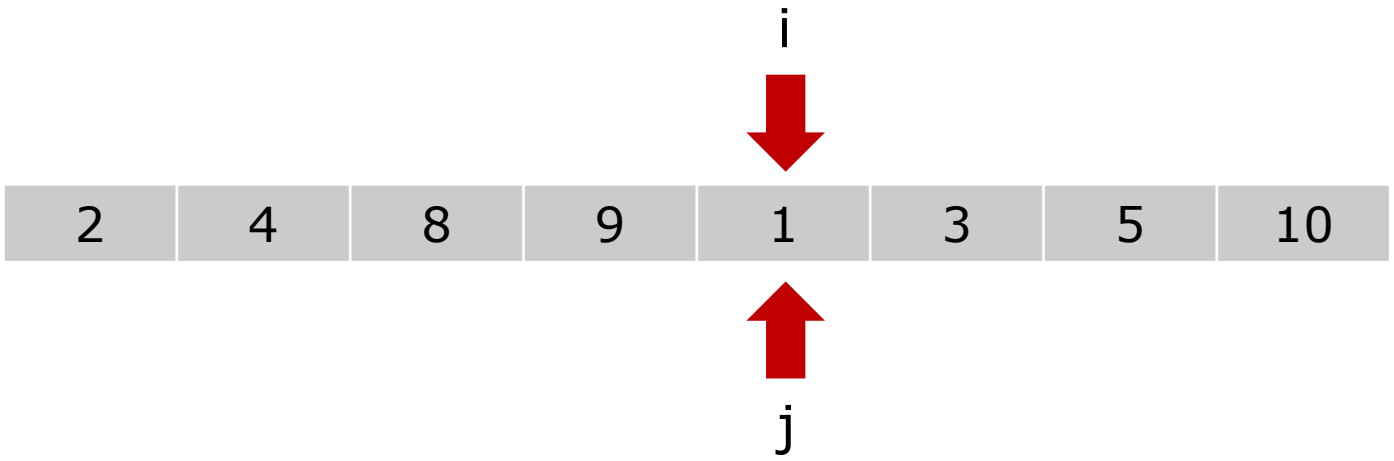
- Algorithm:
  - For i=0,…,n-1:
    - j=i
    - While j>0 AND A[j]<A[j-1]:
      - Swap(A[j],A[j-1])
      - j = j-1

Machine Learning



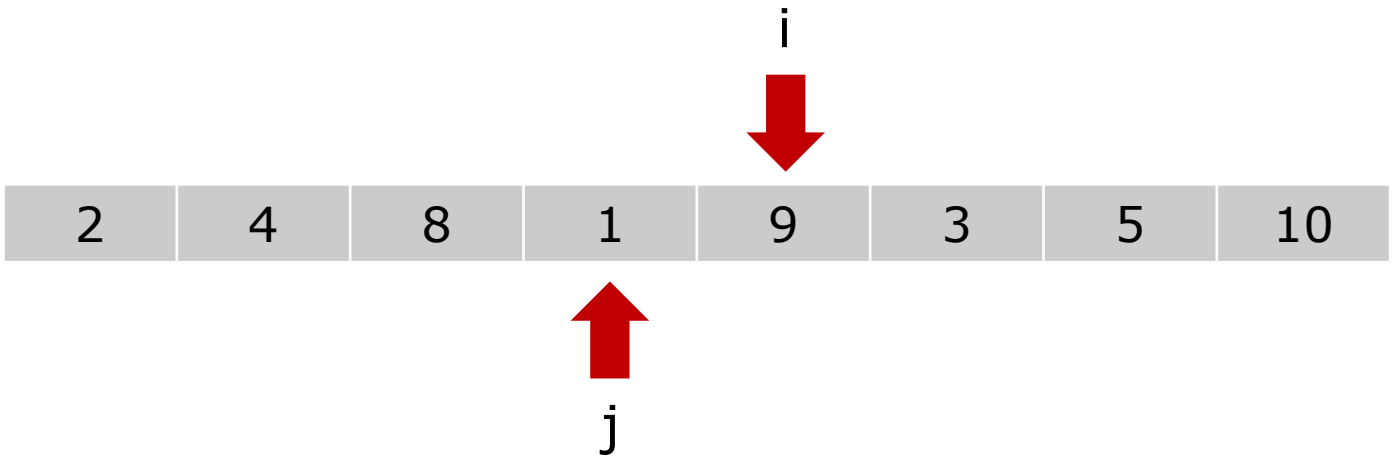
- Algorithm:
  - For i=0,…,n-1:
    - j=i
    - While j>0 AND A[j]<A[j-1]:
      - Swap(A[j],A[j-1])
      - j = j-1

Machine Learning



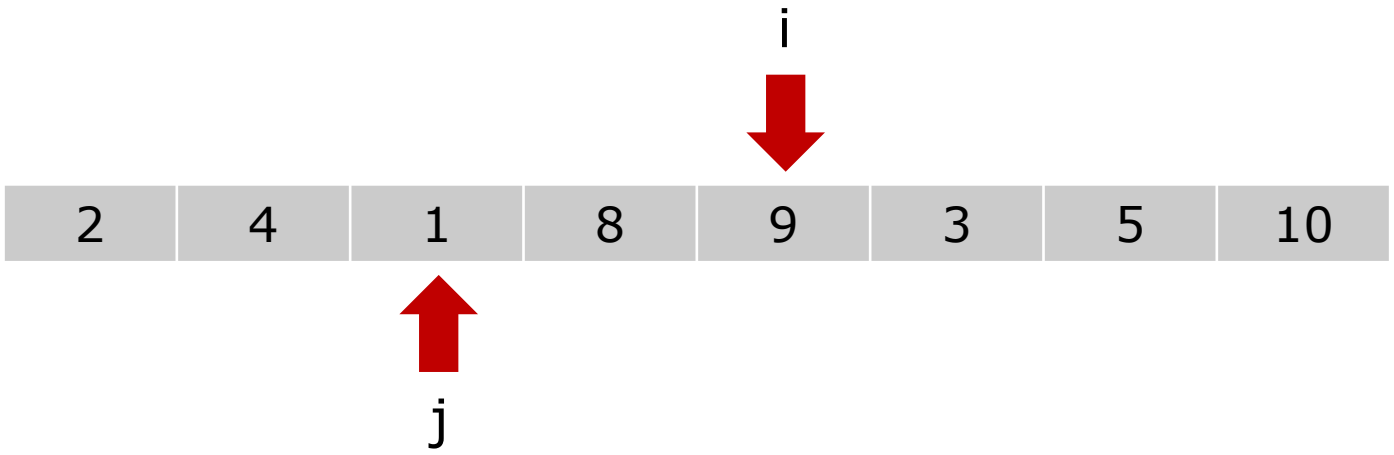
- Algorithm:
  - For i=0,…,n-1:
    - j=i
    - While j>0 AND A[j]<A[j-1]:
      - Swap(A[j],A[j-1])
      - j = j-1

Machine Learning



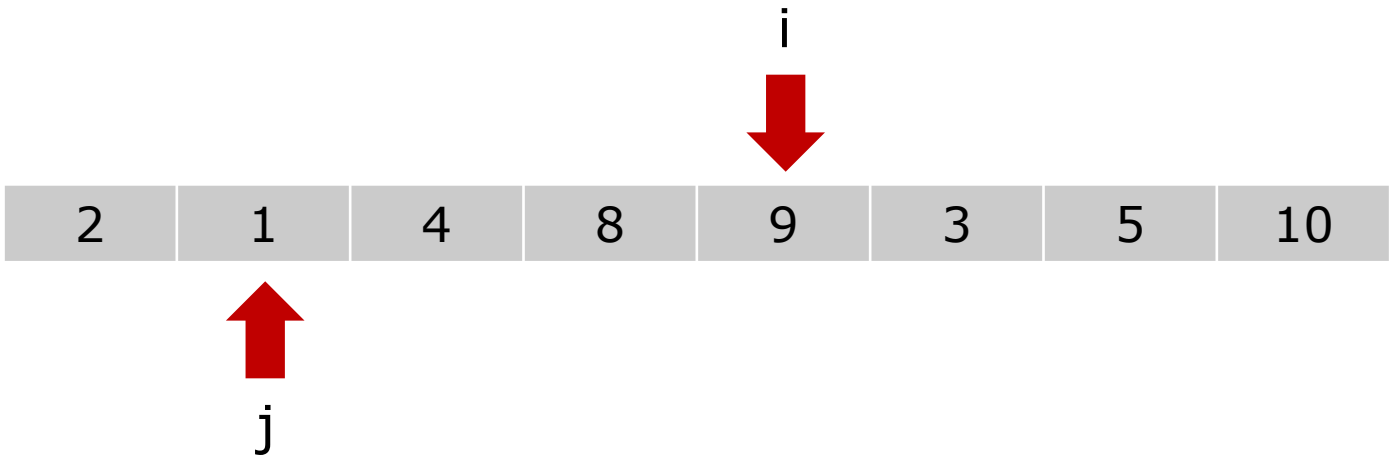
- Algorithm:
  - For i=0,…,n-1:
    - j=i
    - While j>0 AND A[j]<A[j-1]:
      - Swap(A[j],A[j-1])
      - j = j-1

Machine Learning



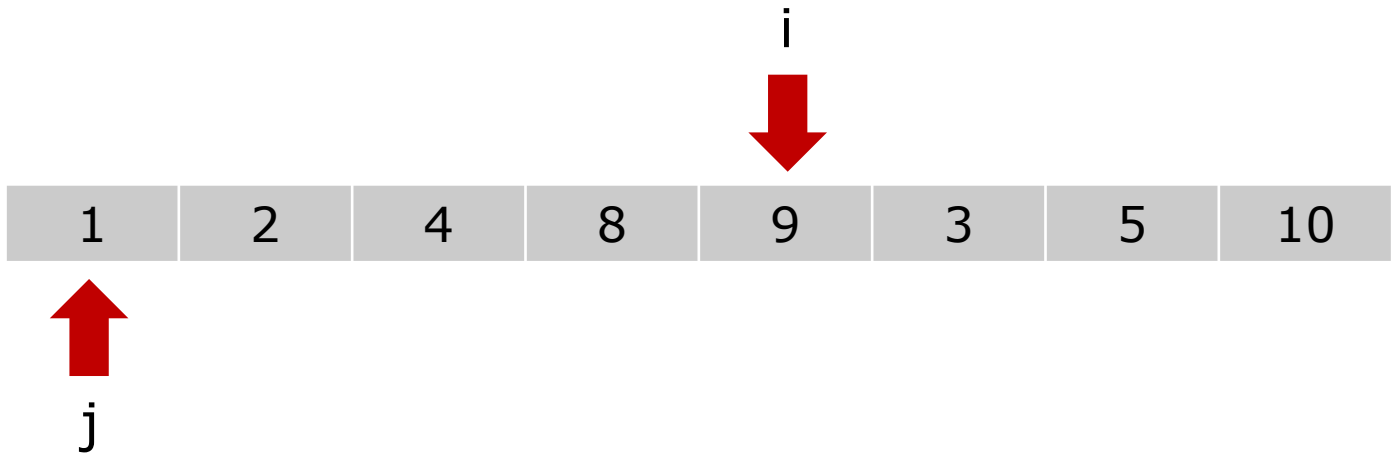
- Algorithm:
  - For i=0,…,n-1:
    - j=i
    - While j>0 AND A[j]<A[j-1]:
      - Swap(A[j],A[j-1])
      - j = j-1

Machine Learning



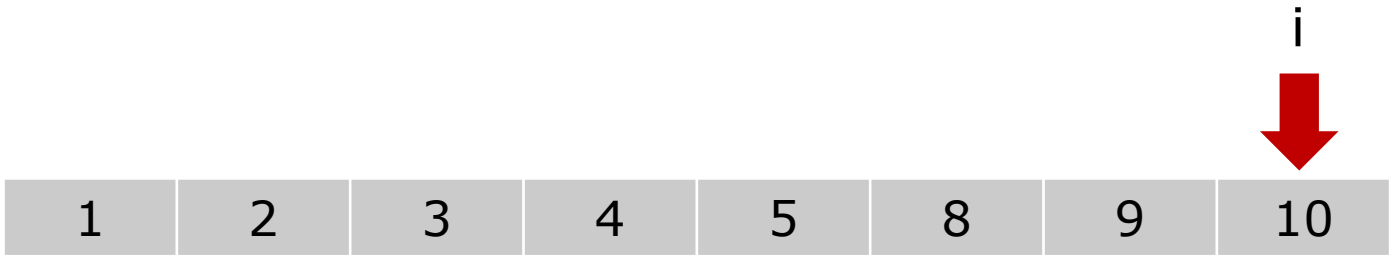
- Algorithm:
  - For i=0,…,n-1:
    - j=i
    - While j>0 AND A[j]<A[j-1]:
      - Swap(A[j],A[j-1])
      - j = j-1

Machine Learning



- Algorithm:
  - For i=0,…,n-1:
    - j=i
    - While j>0 AND A[j]<A[j-1]:
      - Swap(A[j],A[j-1])
      - j = j-1

Machine Learning



- Algorithm:
  - For i=0,…,n-1:
    - j=i
    - While j>0 AND A[j]<A[j-1]:
      - Swap(A[j],A[j-1])
      - j = j-1

Machine Learning

- Classic Algorithm Design
  - Example:
  - Input: Array of n numbers.
  - Output: Array with the numbers sorted.

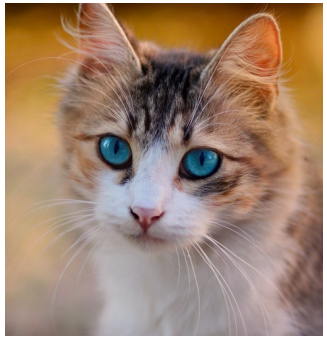
4	2	9	8	1	3	5	10

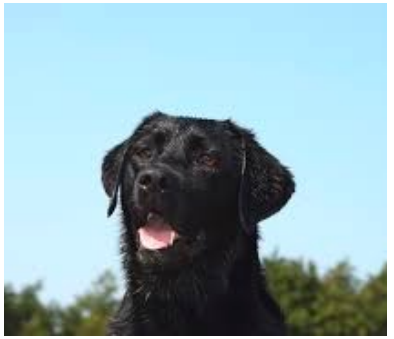
- Algorithm:
  - For i=0,…,n-1:
    - j=i
    - While j>0 AND A[j]<A[j-1]:
      - Swap(A[j],A[j-1])
      - j = j-1

Design algorithms by defining each step.

Machine Learning

- Classic Algorithm Design
  - Sometimes infeasible!
  - Input: Image with 1024x1024 pixels.
  - Output: Is it an image of a cat or a dog.



- Algorithm:
  - If pixel[346][100] is brown AND pixel[592][34] is yellow:
    - Return ”dog”
  - Else if pixel[234][678] is pink:
    - Return ”cat”

Machine Learning

- Classic Algorithm Design
  - Sometimes hard to do well!
  - Input: Properties of a wine (alcohol, ph,…)
  - Output: Is it barollo or a brunello?



- Algorithm:
  - If alcohol>12.4% AND ph<6.7:
    - Return barollo
  - Else if alcohol<11.4% AND…:
    - Return brunello
  - Else if …:

Hard to get right!

Machine Learning

- Classic Algorithm Design
  - Sometimes hard to do well!
  - Input: Properties of a house (m^2, constYear, bedrooms,…)
  - Output: How much is it worth?



Hard to get right!

- Algorithm:
  - CostPrSqMeter = 16.000-constYear*500 + …
  - Return CostPrSqMeter*m^2

Machine Learning

- Learning from data…
  - Said shortly:
    - Algorithms that produce algorithms based on data.
      - Why?
      - How?
  - Why:
    - Sometimes infeasible or hard to find the right/best set of rules/steps.

Machine Learning

- Learning from data…
  - How?
    - The main focus of this course.
    - Before we can start answering, need to talk about the three (main) types of machine learning.
    - Supervised learning
    - Unsupervised learning
    - Reinforcement learning

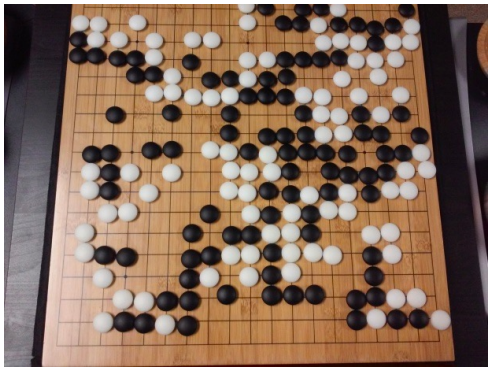
Reinforcement Learning



- Not covered in this course!

- Representative problems:
  - Playing games







- Agent that interacts with environment.
  - Agent is in a state. Can take actions.
  - Actions lead to rewards
  - Goal: Maximize reward

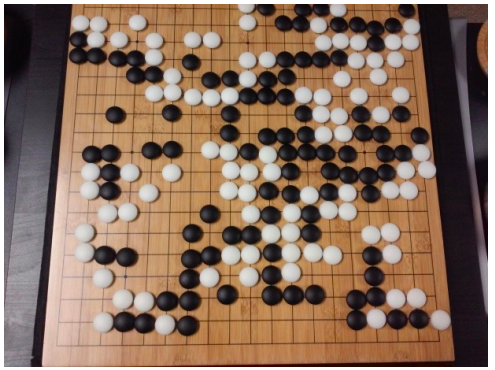
Reinforcement Learning



- Not covered in this course!

- Representative problems:
  - Playing games





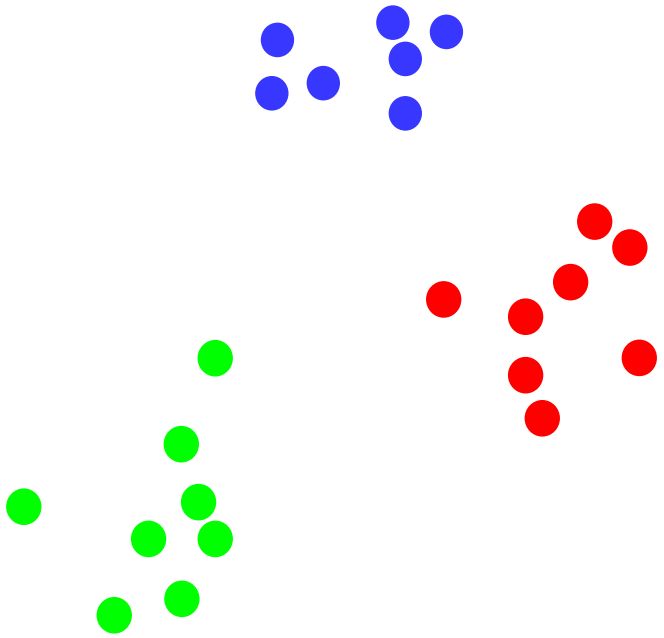


- How is data used?
  - Self-play.
  - Use results of games/actions as ”training” data.
  - Learn a good strategy from the data.

Unsupervised Learning

- Towards the end of this course

- Representative problems:
  - Clustering

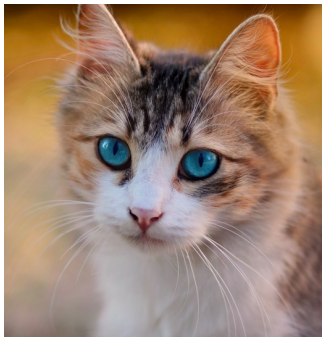


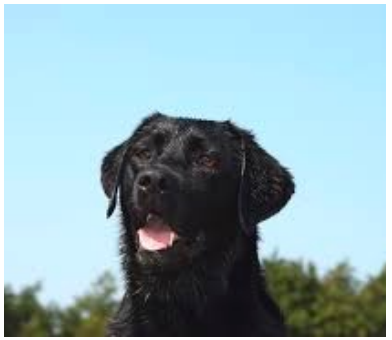
- Partition data into similar groups.
  - Targeted advertising: Partition customers into similar purchase patterns.

Supervised Learning

- Main focus of this course

- Representative problems:
  - Classification and Regression

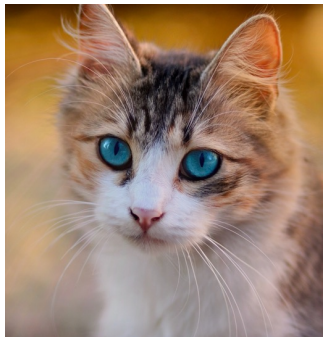


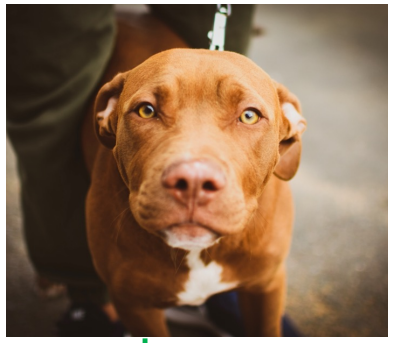



- Learn to classify unknown data based on known training data.
  - Training data with features and labels.
    - Pictures of cats and dogs.
    - Features: Pixels, Labels: {cat, dog}.

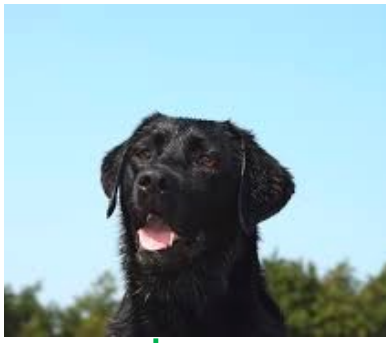
Supervised Learning

- Setup
  - Input: Training images of cats and dogs, with labels.
  - Features: Pixels, Labels: {cat, dog}.


cat


dog


cat


dog

- Predict:
  - Get new picture without label.
  - Predict label.



Cat or dog?

Supervised Learning





- Setup
  - Input: Training wines, with labels.
  - Features: alcohol, ph,… , Labels: {barollo, brunello}.

Alcohol: 12,1% Ph: 5,6 …	Alcohol: 12,4% Ph: 5,6 …	Alcohol: 11,7% Ph: 5,8 …	Alcohol: 11,9% Ph: 6,3 …
Barollo	Barollo	Brunello	Barollo

- Predict:
  - Get new wine without label.
  - Predict label.

Alcohol: 12,0%
Ph: 5,7
…

Barollo or Brunello?

Supervised Learning



- Setup
  - Input: Training houses, with labels.
  - Features: m^2, constYear, bedrooms… , Labels: R^+

m^2: 180 constYear: 1970 …	m^2: 135 constYear: 2003 …	m^2: 210 constYear: 2018 …	m^2: 160 constYear: 2003 …
2,7M	2,2M	5,1M	2,6M

- Predict:
  - Get new house without label.
  - Predict label.

m^2: 170
constYear: 2010
…

Price?

Supervised Learning

- Learn to classify unknown data based on known training data.
  - Training data with features and labels.
    - Pictures of cats and dogs.
    - Features: Pixels, Labels: {cat, dog}.

- How??
  - Focus next many weeks.

Supervised Learning

- How to learn from labeled data.
- Classification.
  - Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$.
    - Each x_i has d features. A vector in R^d.
      - For instance one coordinate per pixel in image.
    - Each y_i is a label from predefined set of labels Y.
      - For instance Y={cat, dog}.
  - Goal: Return a new algorithm A that is good at predicting the label of a new data point x in R^d.
  - Important: When predicting, we only see features. When training, we also see labels.

Supervised Learning

- How to learn from labeled data.
- Classification.
  - Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$.
    - Each x_i has d features. A vector in R^d.
    - Each y_i is a label from predefined set of labels Y.
  - Goal: Return a new algorithm A that is good at predicting the label of a new data point x in R^d.
  - Problem: We have no idea what new point is while training.
    - Probably not in our training data!

Supervised Learning

- How to learn from labeled data.
- Classification.
  - Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$.
    - Each x_i has d features. A vector in R^d.
    - Each y_i is a label from predefined set of labels Y.
  - Goal: Return a new algorithm A that is good at predicting the label of a new data point x in R^d.
  - Hope: New data is "similar" to training data.
    - Therefore, if we can produce A that is correct on most training data, it is hopefully also correct on new data.

Supervised Learning

- A bit more formal.
- Classification.
  - Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$.
    - Each x_i has d features. A vector in R^d.
    - Each y_i is a label from predefined set of labels Y.
  - Unknown target function f that we try to learn.
    - Target function $f : R^d \rightarrow Y$.
    - Training data satisfies $y_i = f(x_i)$.
  - Goal: Return algorithm/hypothesis g such that $g(x_i)=y_i$ as often as possible. Hopefully, $g \approx f$.

Supervised Learning

- A bit more formal.
- Classification.
  - Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$.
    - Each x_i has d features. A vector in R^d.
    - Each y_i is a label from predefined set of labels Y.
  - Unknown target function f that we try to learn.
    - Target function $f : R^d \rightarrow Y$.
    - Training data satisfies $y_i = f(x_i)$.
  - Example: f is the hard-to-define function that correctly maps images to {cat,dog}.

Supervised Learning

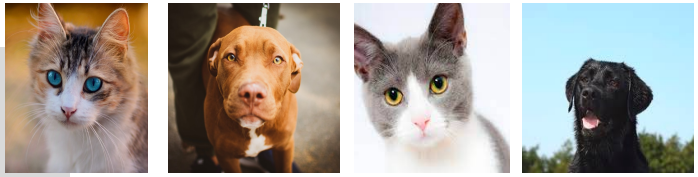
- A bit more formal.
- Classification.
  - Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$.
    - Each x_i has d features. A vector in R^d.
    - Each y_i is a label from predefined set of labels Y.
  - Unknown target function f that we try to learn.
    - Target function $f : R^d \rightarrow Y$.
    - Training data satisfies $y_i = f(x_i)$.
  - Goal: Return algorithm/hypothesis g such that $g(x_i)=y_i$ as often as possible. Hopefully, $g \approx f$.

Supervised Learning

UNKNOWN TARGET FUNCTION
$f : R^d \rightarrow Y$

f: Mapping of images to {cat,dog}.

TRAINING EXAMPLES
$(x_1,f(x_1)),\ldots,(x_n,f(x_n))$



cat dog cat dog

LEARNING ALGORITHM
A

FINAL HYPOTHESIS
$g \approx f$

Find g in H with $g(x_i)=f(x_i)$ often.

HYPOTHESIS SET
H

??

The Hypothesis Set

- What is H?
  - Set of candidate algorithms/hypotheses that our learning algorithm may choose from.
  - We will see different H for different learning algorithms.

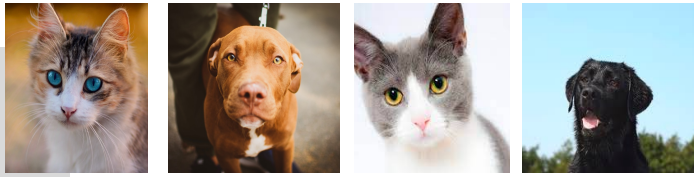
- Simple (but unreasonable) example:
  - H is all functions implementable as Python programs with less than 1,000 characters.

Supervised Learning

UNKNOWN TARGET FUNCTION
$f : R^d \rightarrow Y$

f: Mapping of images to {cat,dog}.

TRAINING EXAMPLES
$(x_1,f(x_1)),\ldots,(x_n,f(x_n))$



cat dog cat dog

LEARNING ALGORITHM
A

FINAL HYPOTHESIS
$g \approx f$

Find g in H with $g(x_i)=f(x_i)$ often.

One approach: Try all g. (SLOW).

HYPOTHESIS SET
H

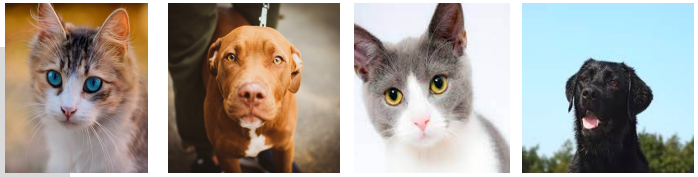
All Python progs. < 1,000 chars.

Supervised Learning

UNKNOWN TARGET FUNCTION
$f : R^d \rightarrow Y$

f: Mapping of images to {cat,dog}.

TRAINING EXAMPLES
$(x_1,f(x_1)),\ldots,(x_n,f(x_n))$



cat dog cat dog

LEARNING ALGORITHM
A

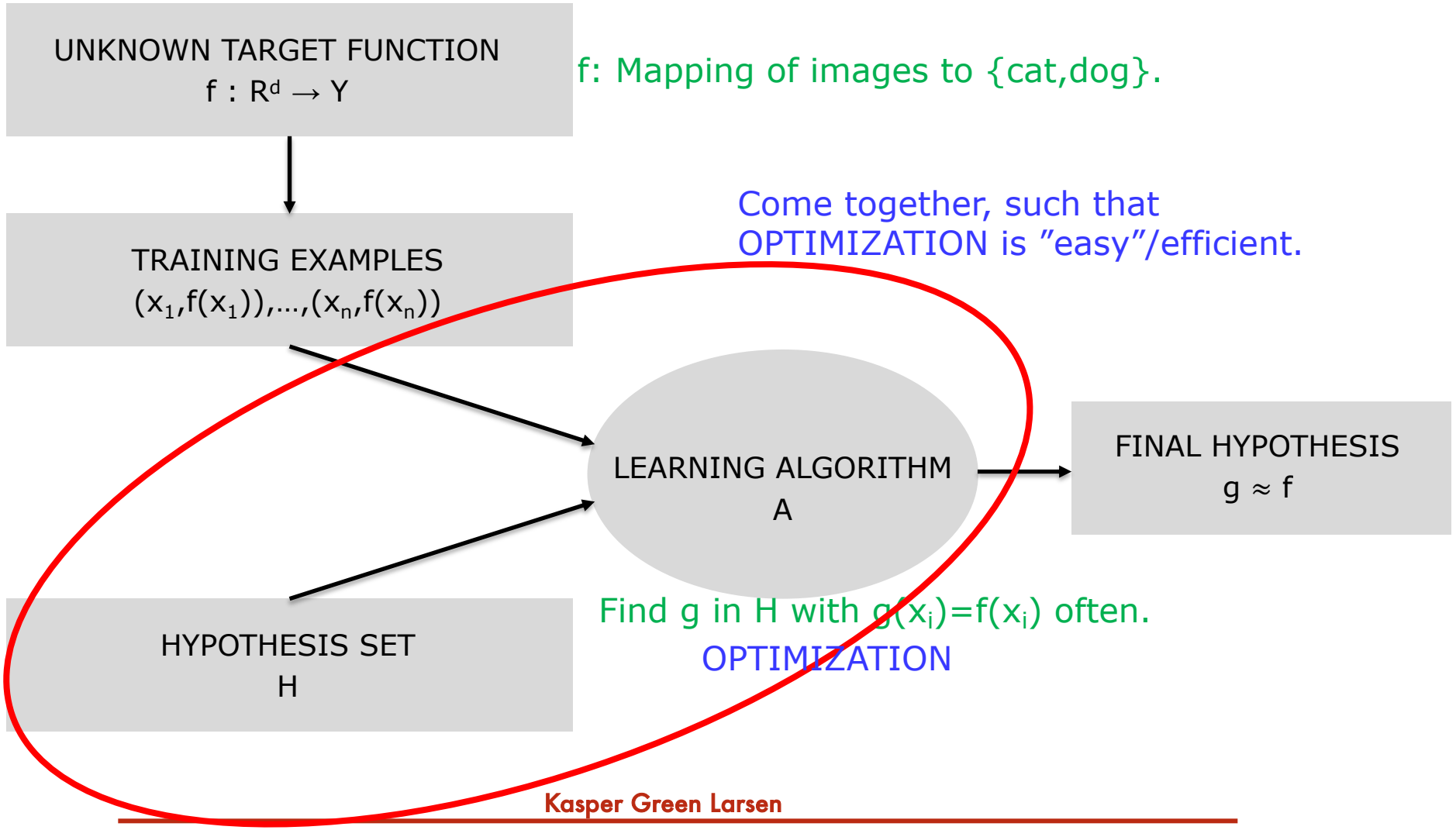
FINAL HYPOTHESIS
$g \approx f$

Find g in H with $g(x_i)=f(x_i)$ often.
OPTIMIZATION

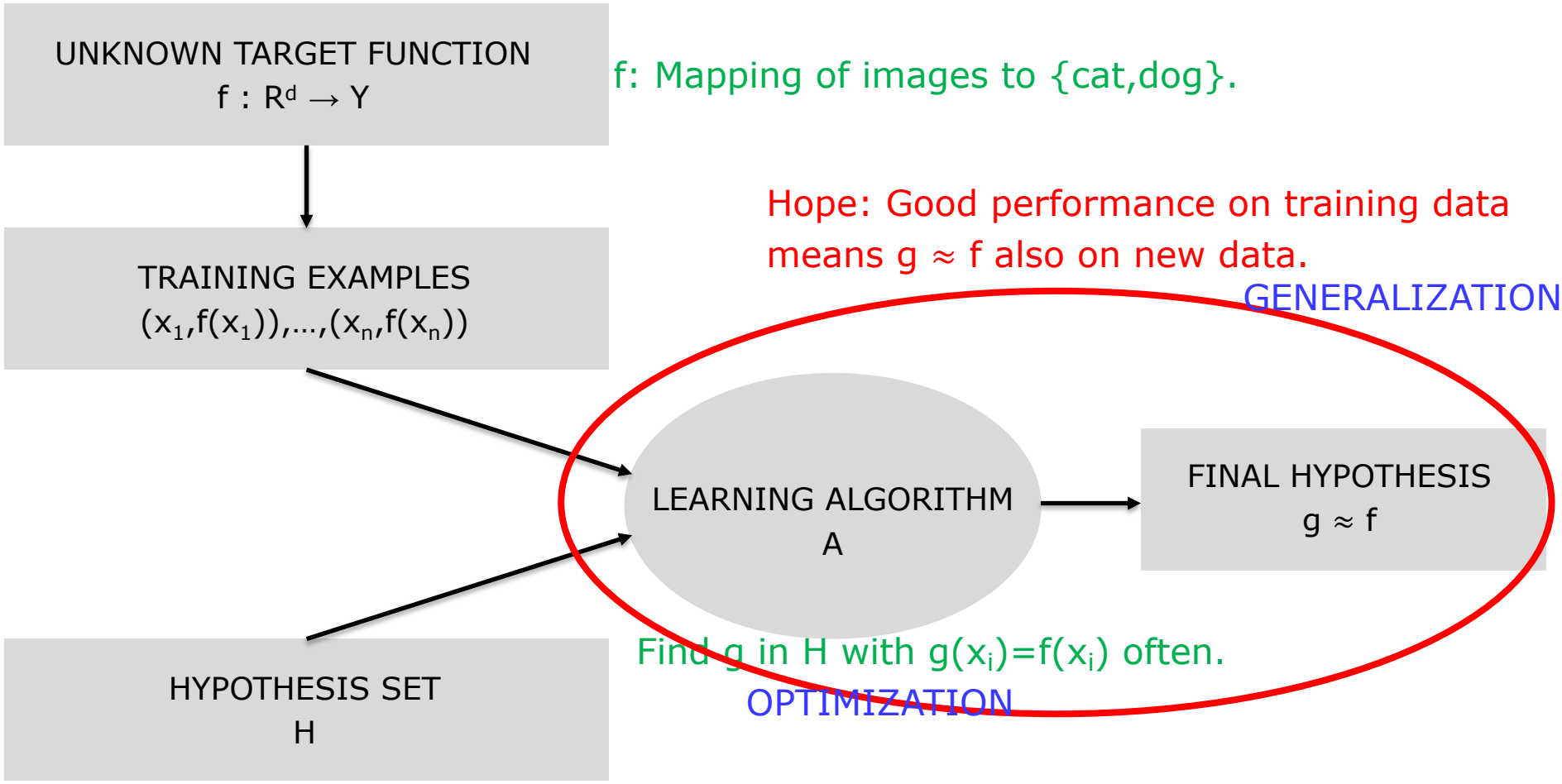
HYPOTHESIS SET
H

All Python progs. < 1,000 chars.

Supervised Learning



Supervised Learning



The Hypothesis Set

The learning algorithm's task to find g.

- Optimization:
  - Choose H such that optimization is efficient.
    - i.e. finding g with $g(x_i)=f(x_i)$ for most i is easy.
  - Need H to be expressive enough such that there is a g in H with $g \approx f$.

- Generalization:
  - Hope if $g(x_i)=f(x_i)$ for most i, then $g \approx f$ also on new data.
  - Learning Theory (coming later):
    - Prove this under statistical assumptions.
    - Main message: The less expressive H is, the more likely this is.
- Tradeoff between optimization and generalization.

The Classifier Interface

- A Classifier as a Class (Object Oriented Programming).

- Two main methods:

- Fit(X,y): X is a matrix with training data as rows.
  - On data $(x_1,y_1),\ldots,(x_n,y_n)$ then X has n rows with i'th row equal to x_i. So n x d matrix when d features.
  - The vector y has i'th coordinate y_i.
  - Method searches for g with $g(x_i)=y_i$ for most i. Stores g.

- Predict(x): x is a vector of d features.
  - Evaluate and return g(x).

Supervised Learning

- We need to see an example!

- Coming Up:
  - Perceptron algorithm
  - First example of a learning algorithm.

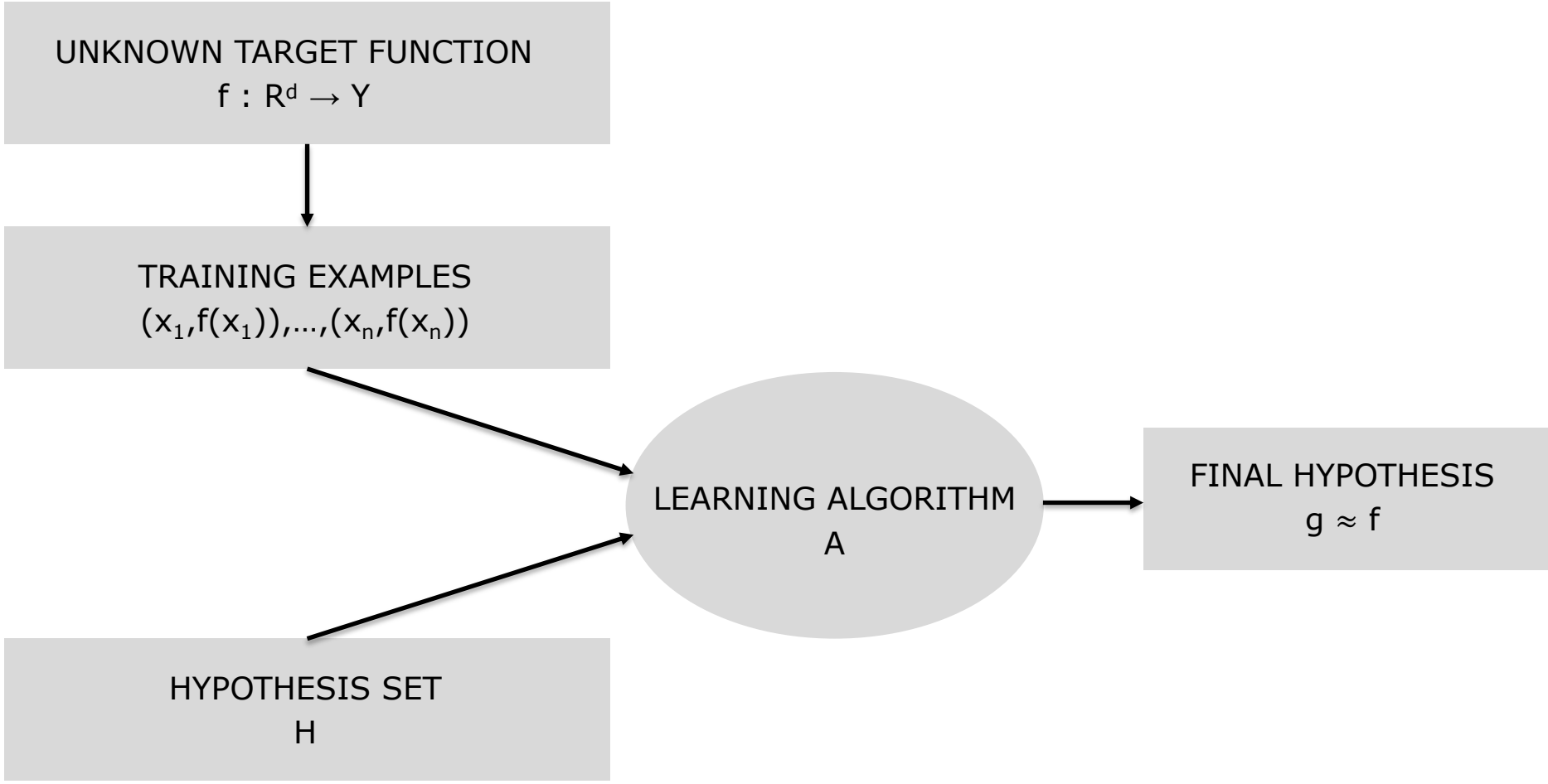
Machine Learning

The Perceptron

Supervised Learning

- Perceptron Algorithm
  - Our first example of learning algorithm.

Supervised Learning



Perceptron

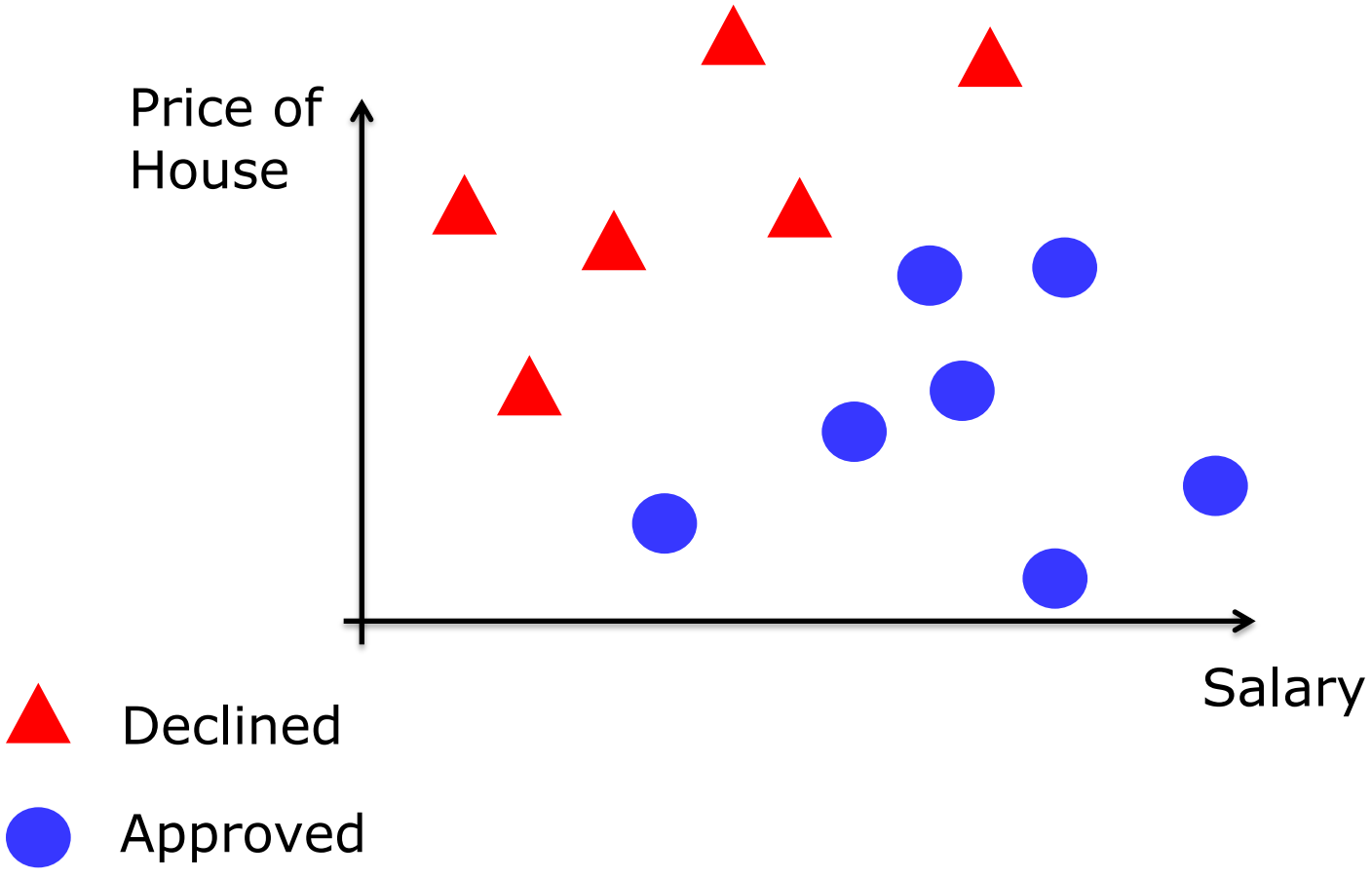
- Example Problem:
  - Mortgage Approval (buy a house).
  - Features: Salary, Debt, Price of house,…
  - Labels: Approve/Decline

- Have historic data of mortgage applications and the outcomes.
- Use this *training data* to find algorithm/hypothesis to automate mortgage approval for new applications.

- For ease of illustration:
  - Let us assume only two features (easier to draw in 2D).
  - Features: Salary, Price of house

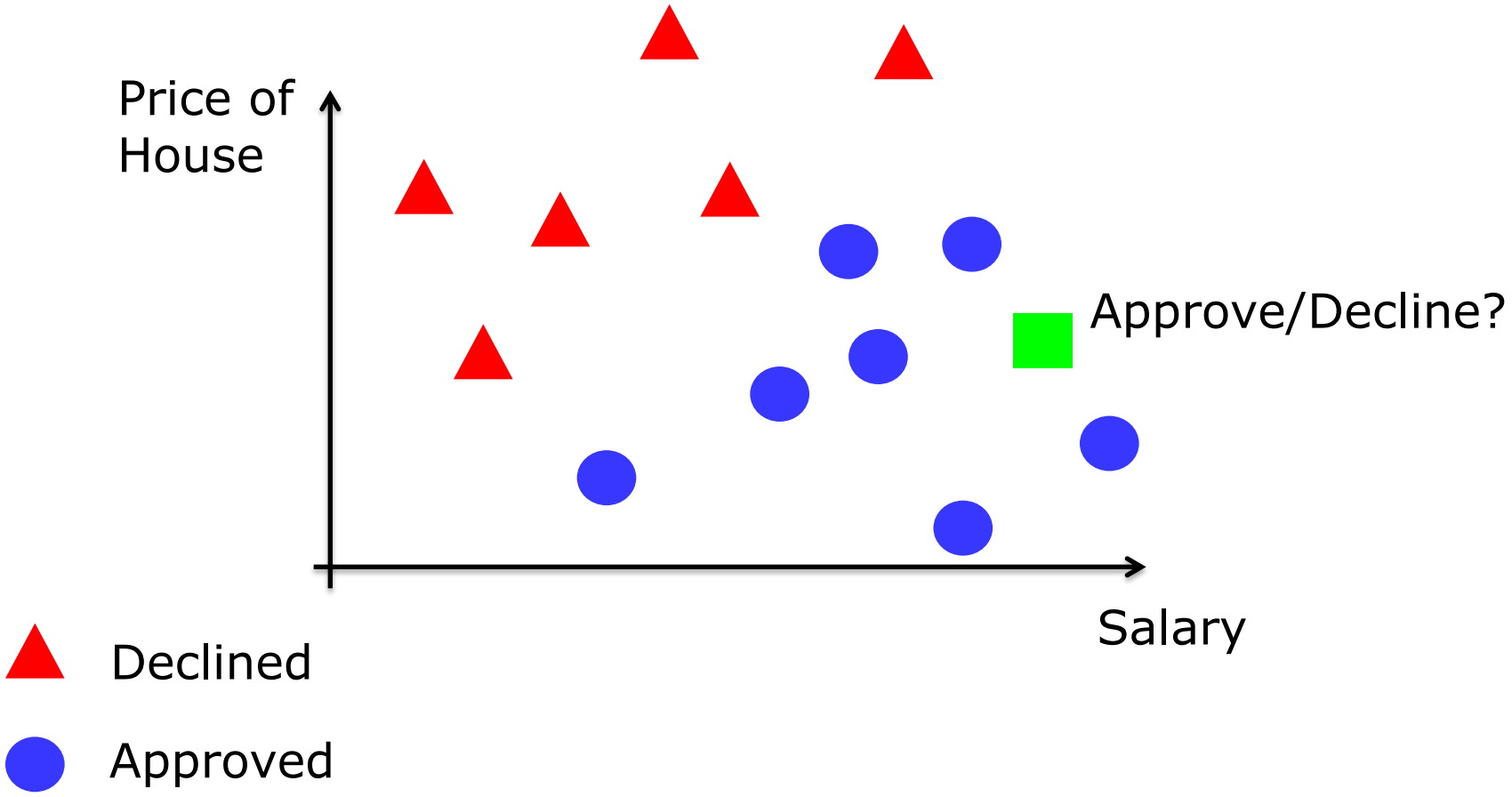
Perceptron

- Training Data:



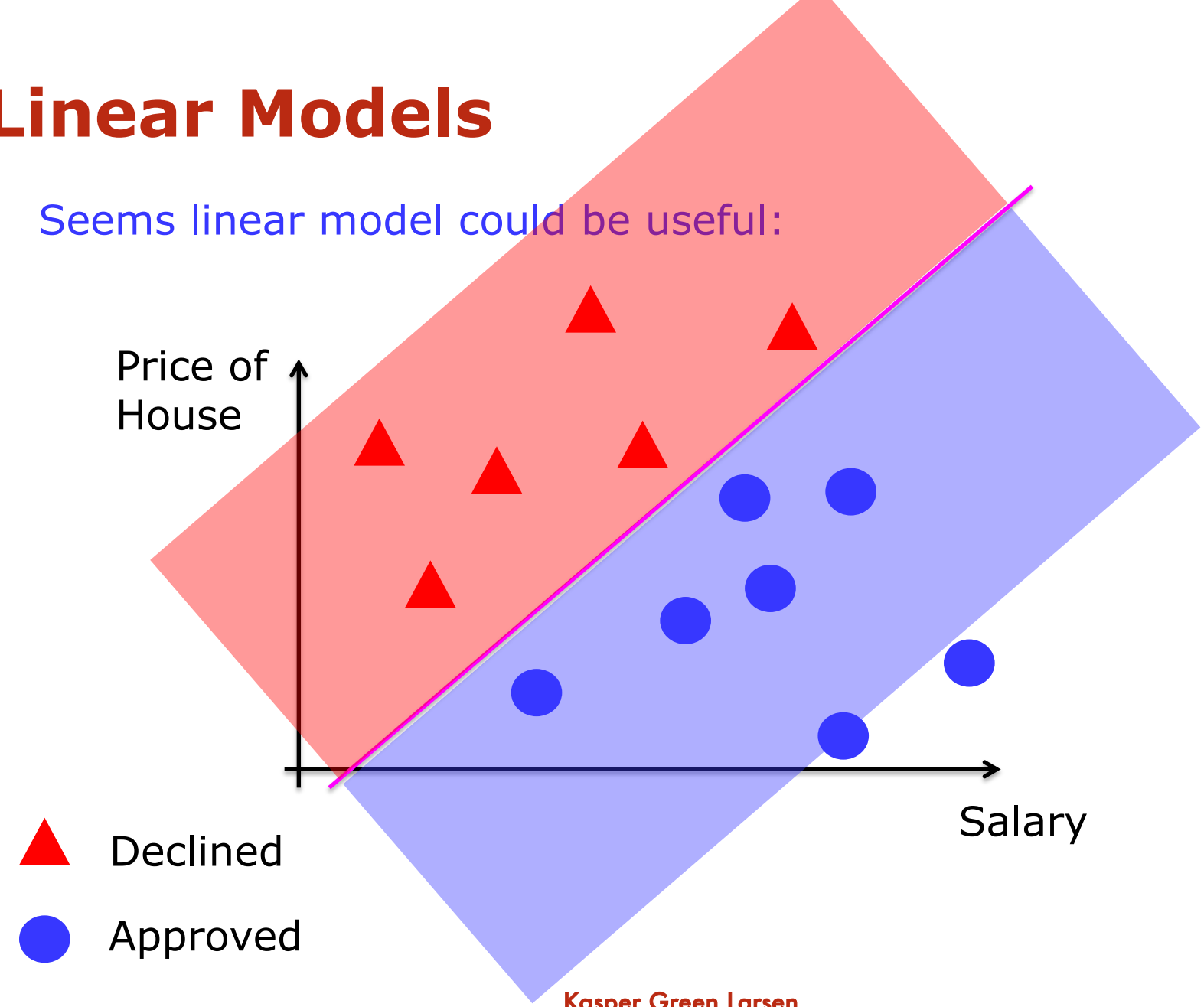
Perceptron

- New data without label.



Linear Models

- Seems linear model could be useful:



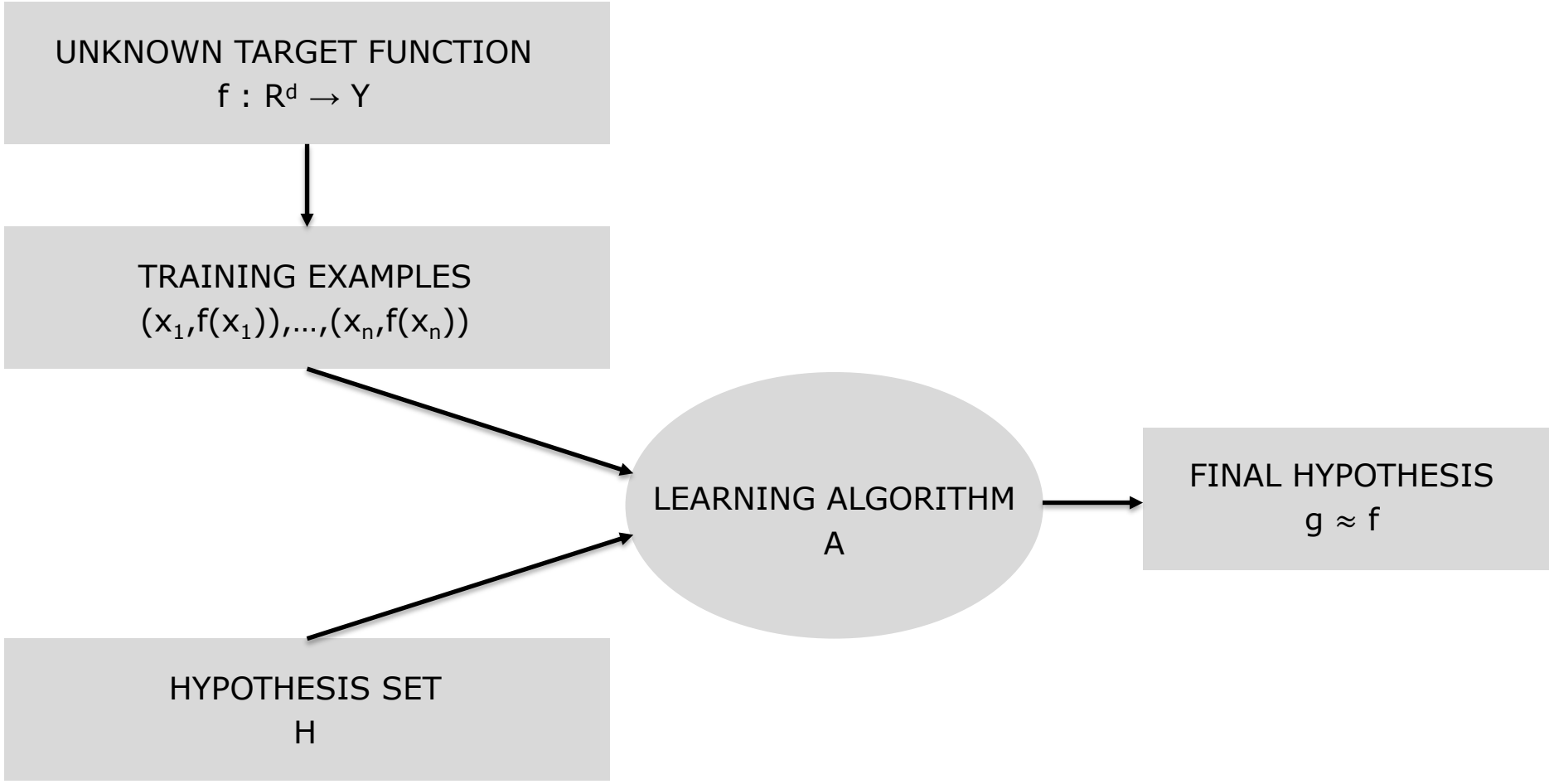
Linear Models

- Mortgage:
  - Features: Salary, Price of house.
  - Labels: Approve/Decline.

- Linear Model:
  - Hypothesis $w = (w_1, w_2, b)$
  - Data features (x_1, x_2)
  - If $w_1x_1 + w_2x_2 > b$: Approve
  - Else: Decline.

  - b is called the *bias*.

Supervised Learning



H=all linear models $(w_1,w_2,\ldots,w_d,b)$

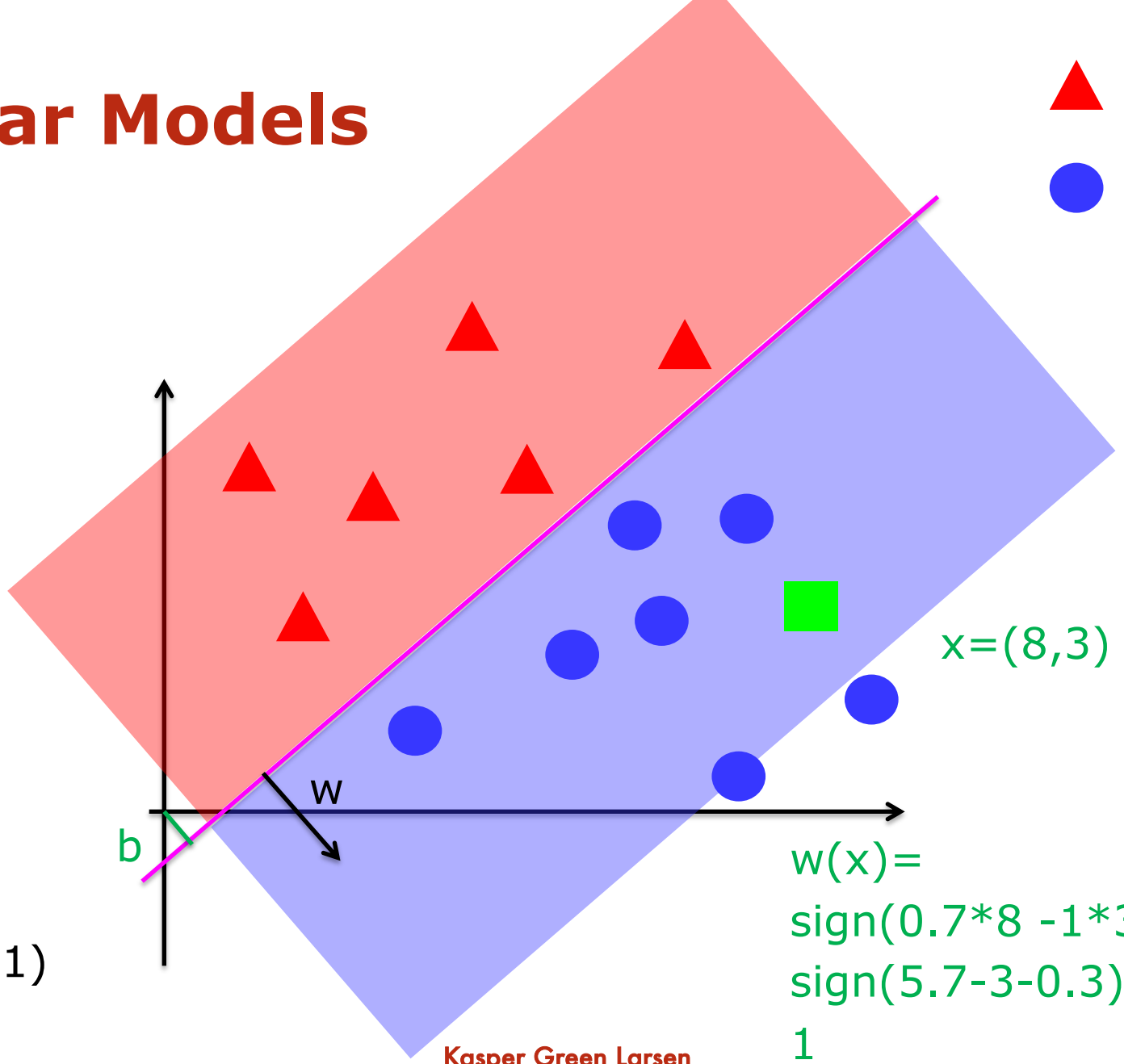
Linear Models

- Binary Classification:
  - Features: $x=(x_1,\ldots,x_d)$
  - Labels: $\{-1,1\}$

- Linear Model:
  - Parameters of hypothesis w: $(w_1, w_2,\ldots, w_d, b)$
  - If $w_1x_1 + w_2x_2 + \ldots + w_dx_d > -b$: Approve
  - Else: Decline.

  - Equivalently:
  - $w(x) = \text{sign}(w_1x_1 + w_2x_2 + \ldots + w_dx_d + b)$

Linear Models



w=(0.7 , -1)

b=-0.3

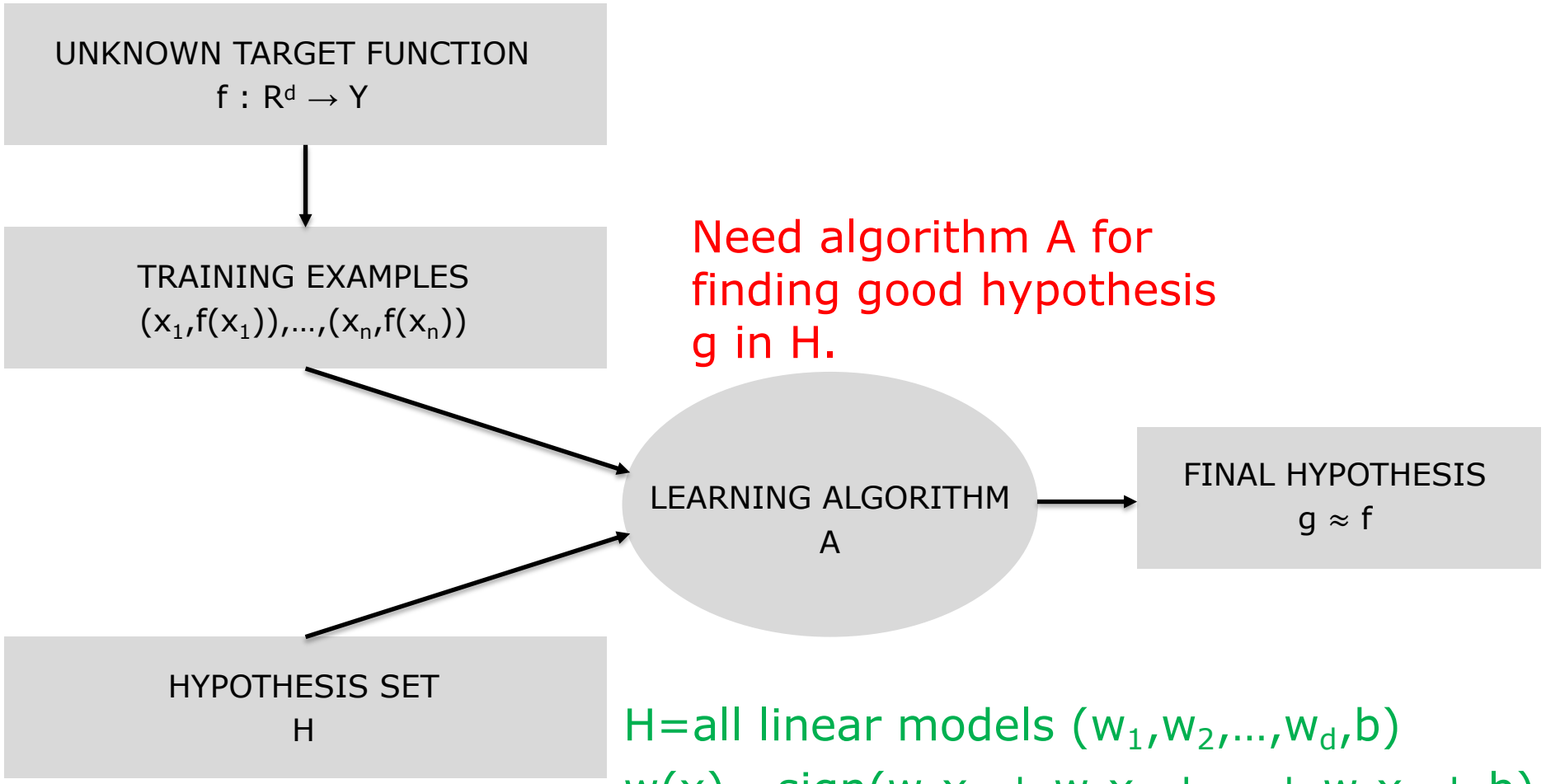
w(x)=
sign(0.7*8 -1*3 -0.3)=
sign(5.7-3-0.3) =
1

Linear Models

- Binary Classification:
  - Features: $x=(x_1,\ldots,x_d)$
  - Labels: $\{-1,1\}$

- Linear Model:
  - Parameters of hypothesis w: $(w_1, w_2,\ldots, w_d, b)$
  - Predict: $w(x) = \text{sign}(w_1x_1 + w_2x_2 + \ldots + w_dx_d + b)$

Supervised Learning



Perceptron Learning Algorithm (PLA)

- Binary Classification:
  - Features: $x=(x_1,\ldots,x_d)$
  - Labels: $\{-1,1\}$

- Linear Model:
  - Parameters of hypothesis w: $(w_1, w_2,\ldots, w_d, b)$
  - Predict: $w(x) = \text{sign}(w_1x_1 + w_2x_2 + \ldots + w_dx_d + b)$

- Perceptron:
  - Algorithm for finding a *good* set of parameters $w_1,\ldots,w_d,b$.
  - Good: Classifies (most of) training data correctly.
  - Recall: Hope good performance on training data means good performance on new data.

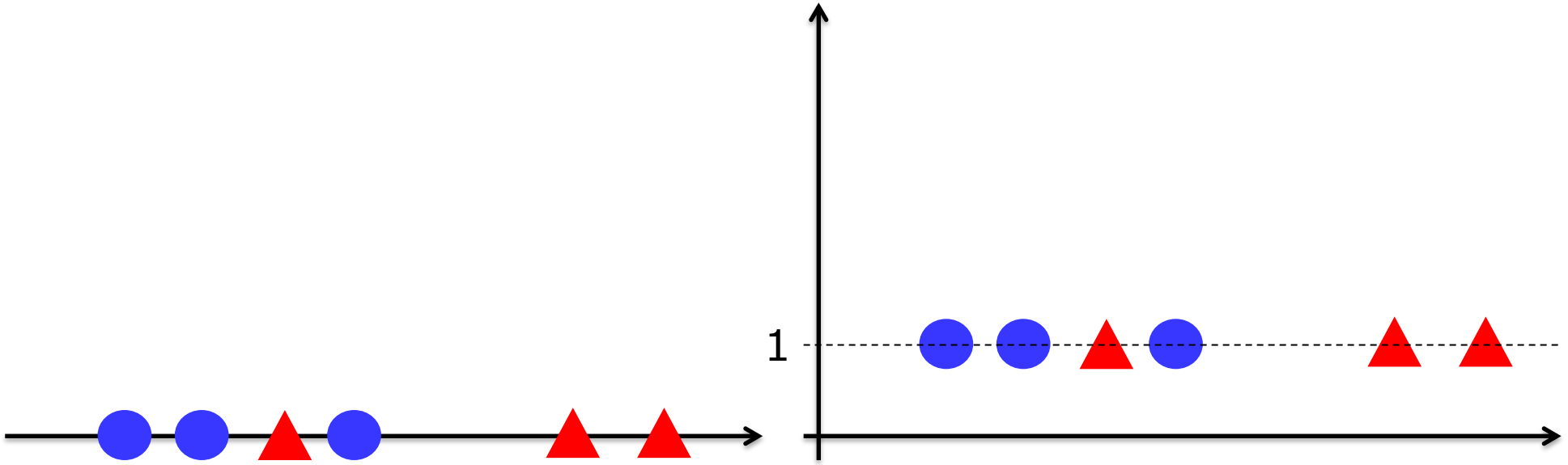
One More Trick

- Binary Classification:
  - Features: $x=(x_1,\ldots,x_d)$
  - Labels: $\{-1,1\}$
- Linear Model:
  - Parameters of hypothesis w: $(w_1, w_2,\ldots, w_d, b)$
  - Predict: $w(x) = \text{sign}(w_1x_1 + w_2x_2 + \ldots + w_dx_d + b)$

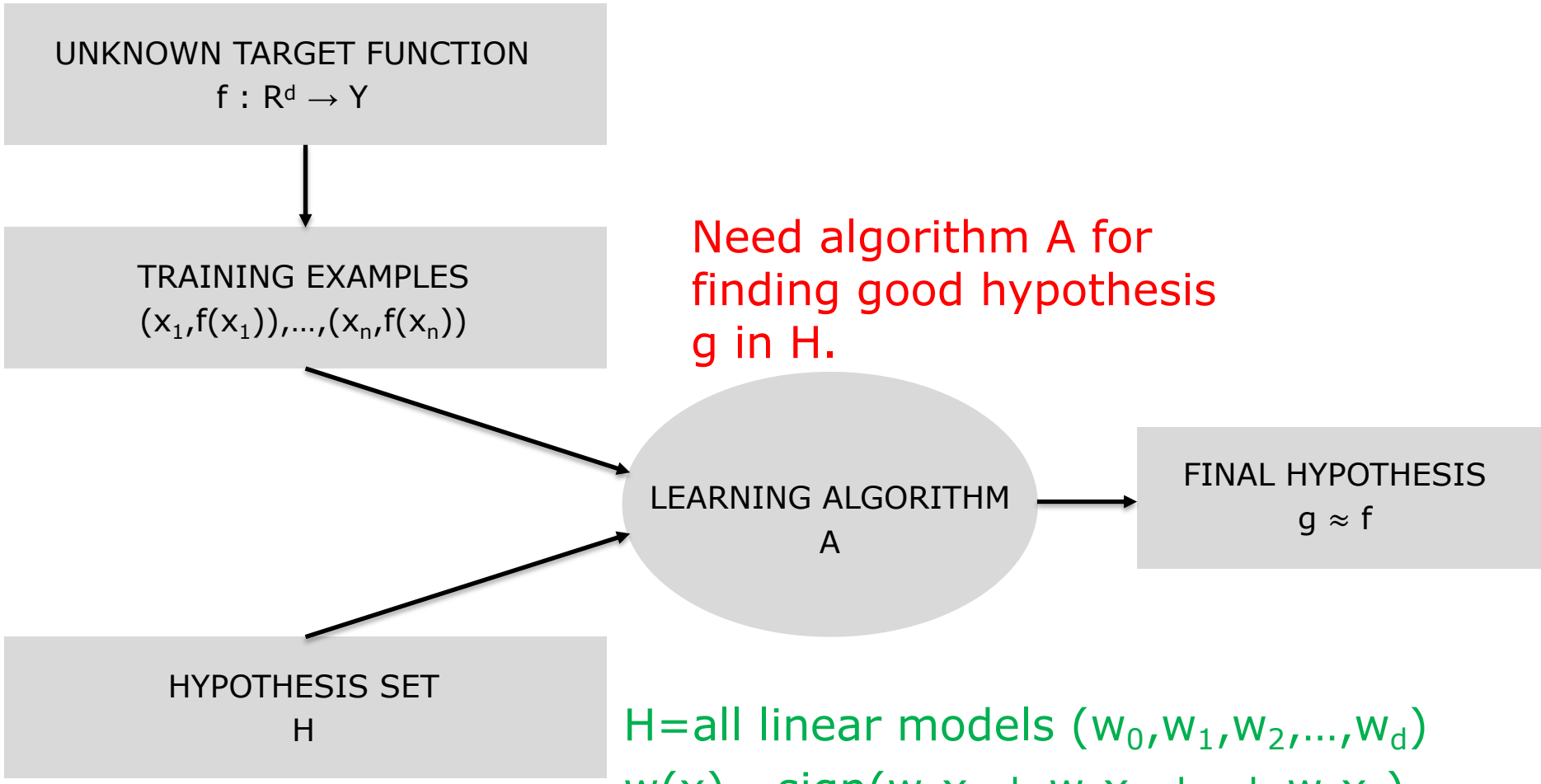
- Trick:
  - Annoying to have special bias parameter b.
  - Make data (d+1)-dimensional and set first coordinate of all data points equal to 1.
  - Then parameters $(w_0, w_1, w_2,\ldots, w_d)$.
  - Predict $w(x) = \text{sign}(w_0 1 + w_1x_1 + w_2x_2 + \ldots + w_dx_d)$.

Trick

- 1-dimensional to 2-dimensional



Supervised Learning



Perceptron Learning Algorithm (PLA)

- Binary Classification:
  - Features: $x=(1,x_1,\ldots,x_d)$
  - Labels: $\{-1,1\}$

Assume w and x column vectors then:

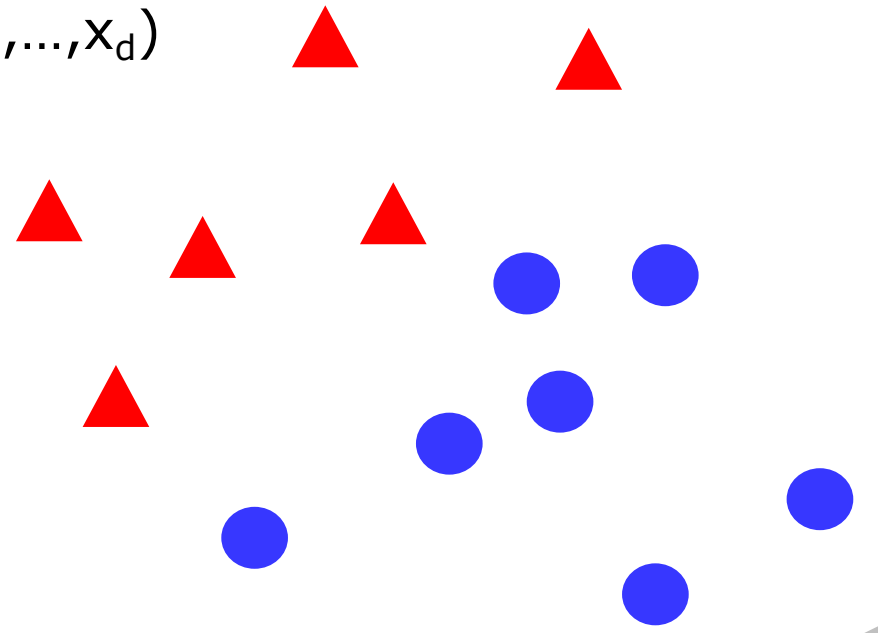
Inner product

- Linear Model:
  - Parameters of hypothesis w: $(w_0, w_1,\ldots, w_d)$
  - Predict: $w(x) = \text{sign}(w_0 1 + w_1 x_1 + \ldots + w_d x_d) = \text{sign}(w^T x)$

- Perceptron:
  - Algorithm for finding a *good* set of parameters $w_0,\ldots,w_d$.
  - Good: Classifies (most of) training data correctly.
  - Recall: Hope good performance on training data means good performance on new data.

Perceptron Learning Algorithm (PLA)

- Surprisingly simple algorithm!
  - Features: $x=(x_0,x_1,\ldots,x_d)$
  - Labels: {-1,1}



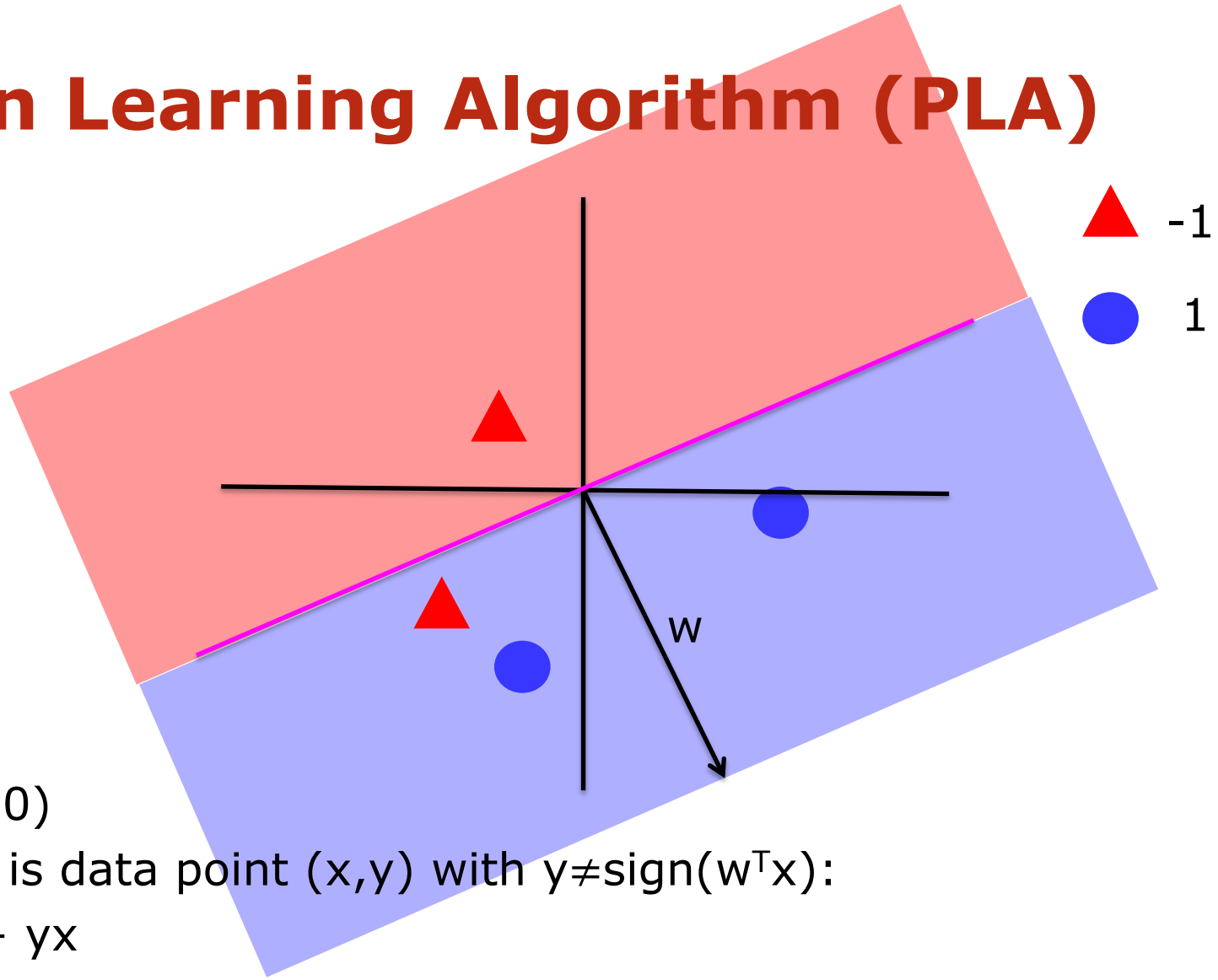
- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Wrong classification

Make w "more" correct on (x,y)

Perceptron Learning Algorithm (PLA)

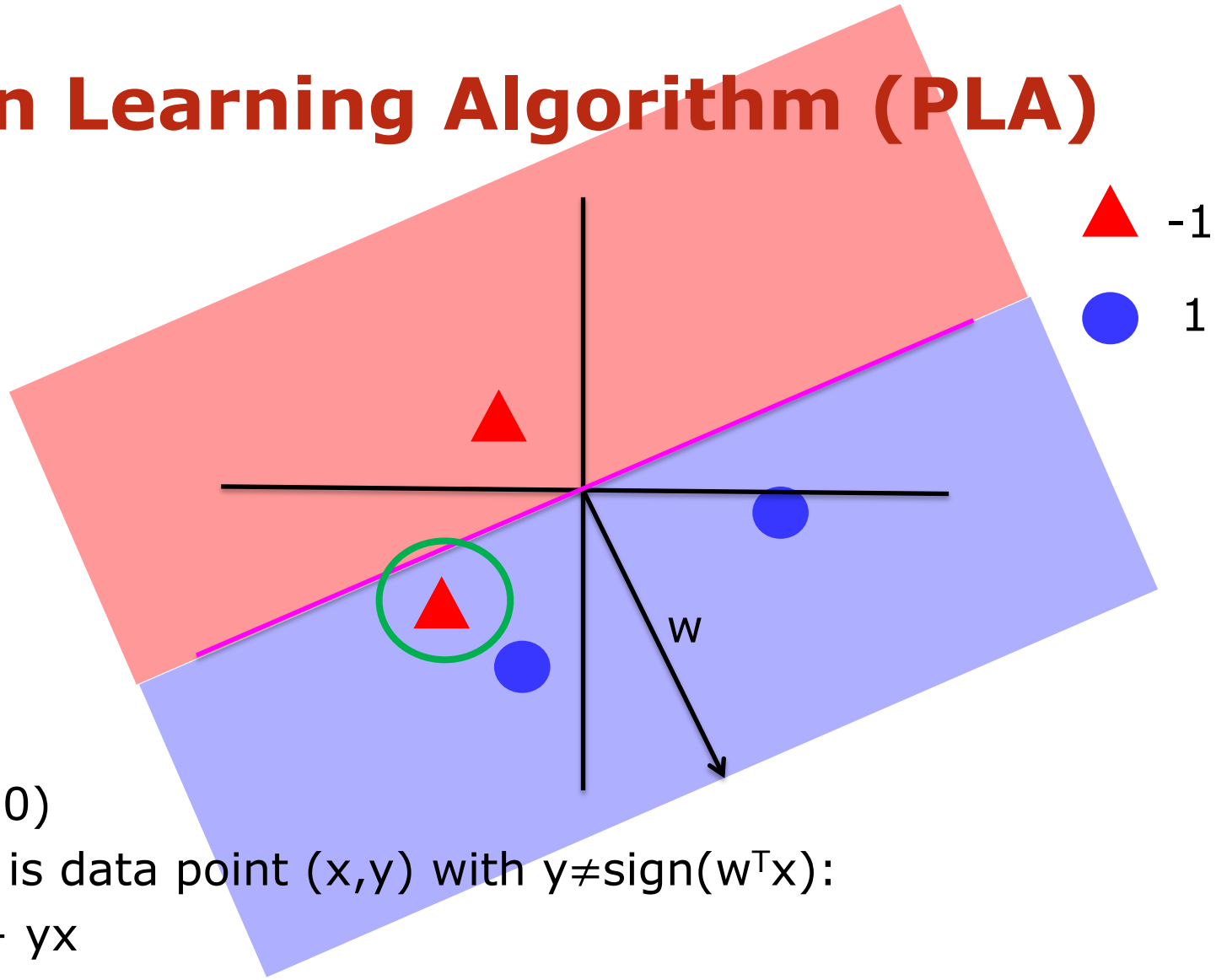
- Example:



- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Perceptron Learning Algorithm (PLA)

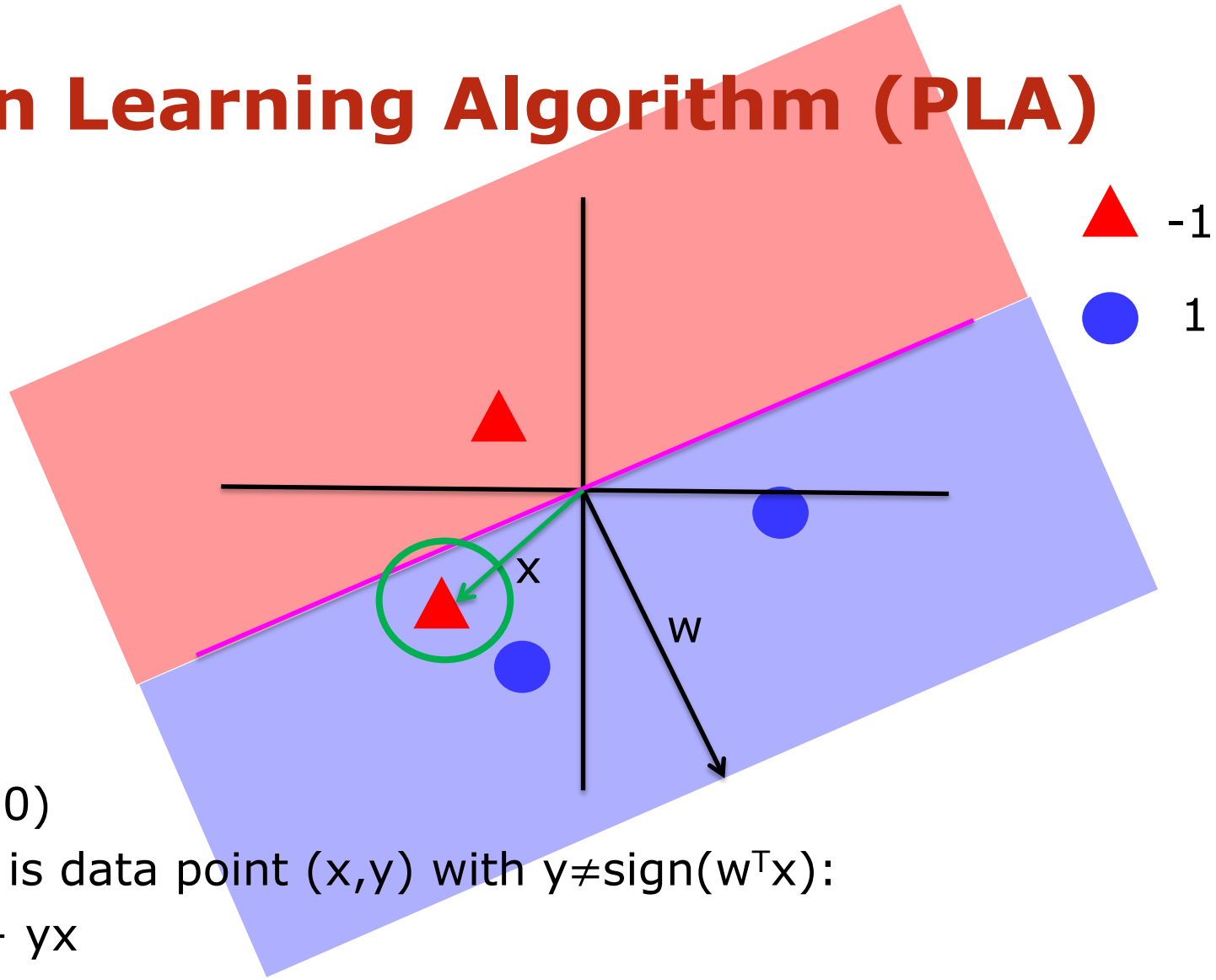
- Example:



- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Perceptron Learning Algorithm (PLA)

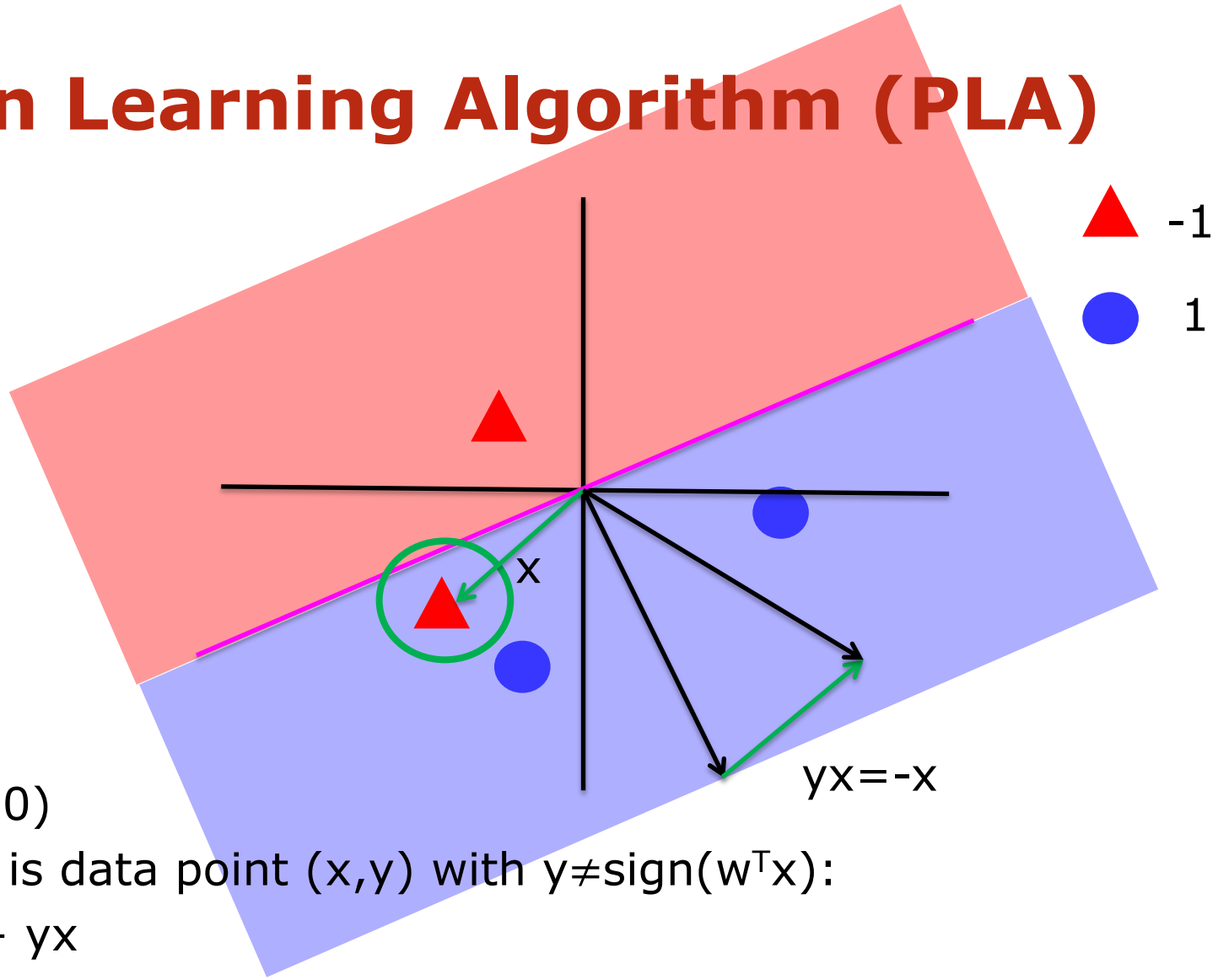
- Example:



- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Perceptron Learning Algorithm (PLA)

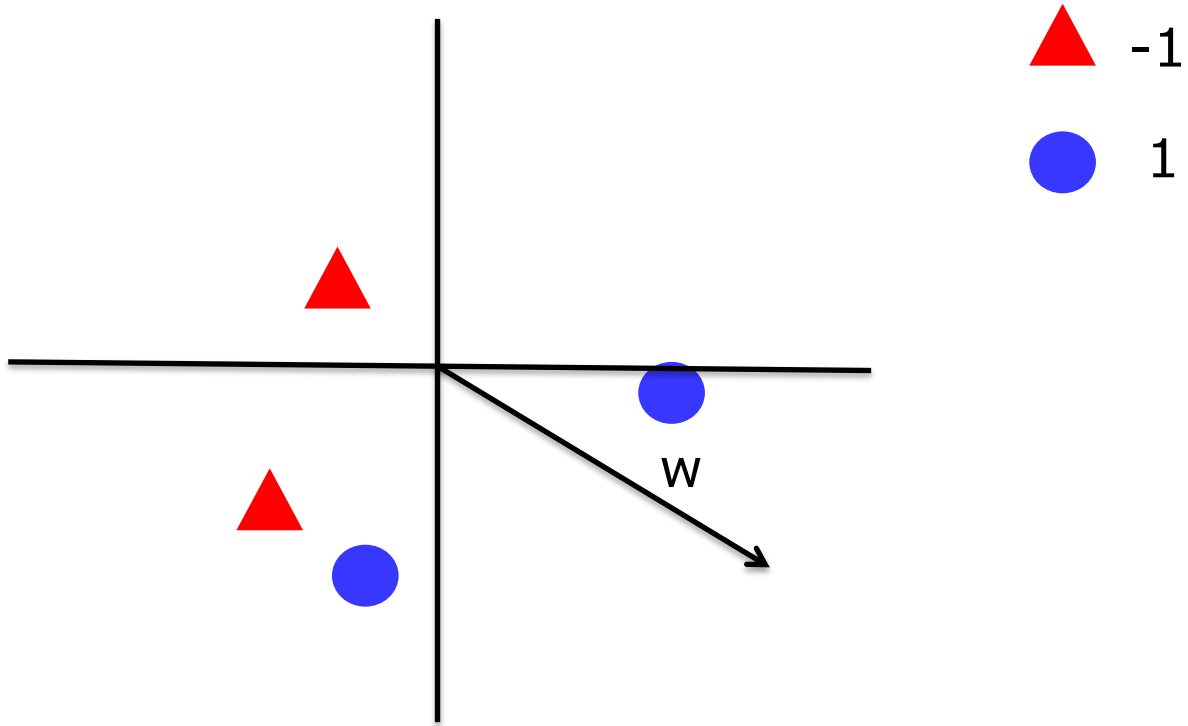
- Example:



- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Perceptron Learning Algorithm (PLA)

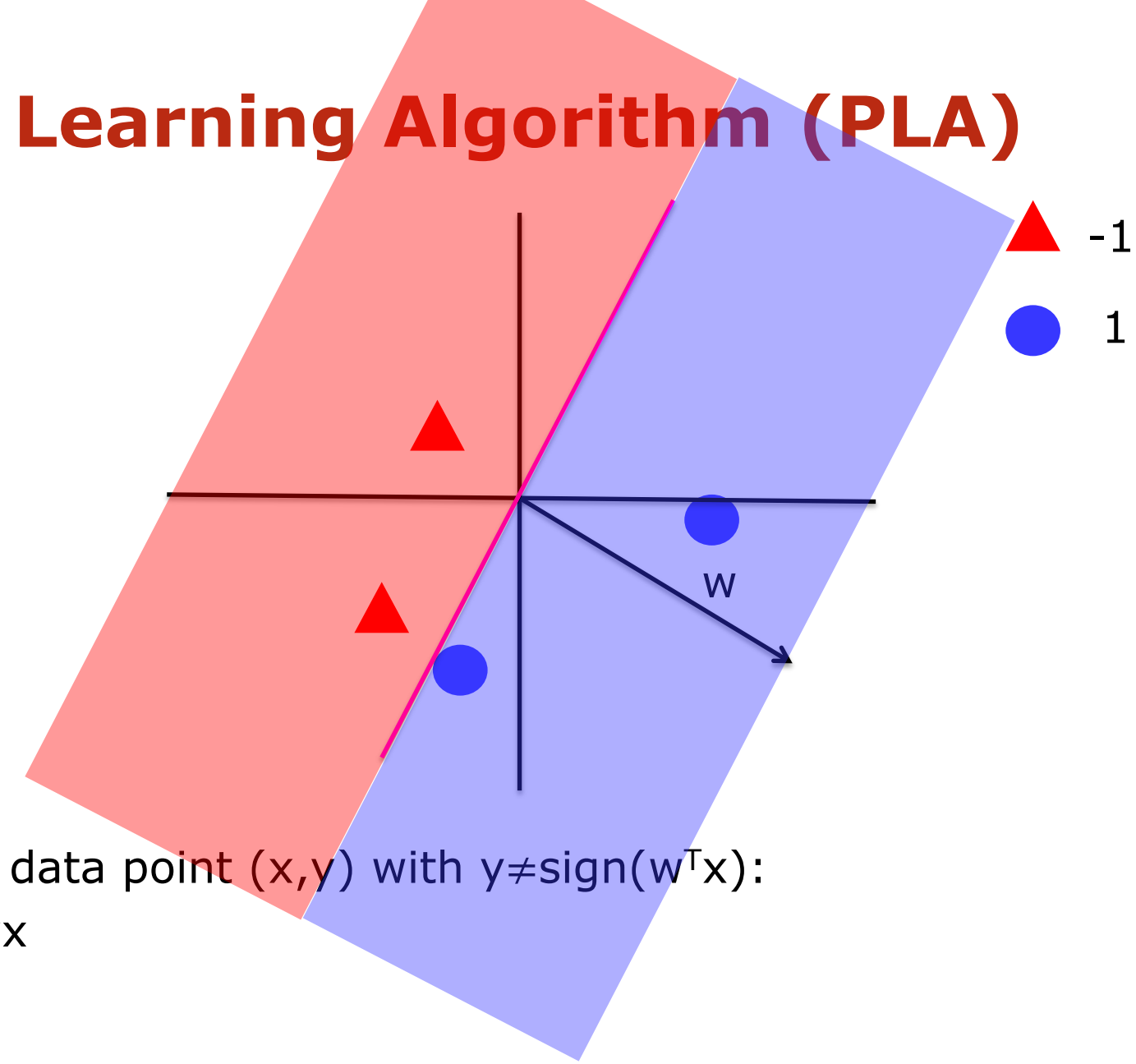
- Example:



- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Perceptron Learning Algorithm (PLA)

- Example:



- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Perceptron Learning Algorithm (PLA)

- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Make w "more" correct on (x,y)

- Why more correct?
  - Let w(t) be w at step t of the while loop.
  - Let x(t) and y(t) be (x
ile loo
  - w(t+1)=w(t)+y(t)x(t)

Value at previous step

$x(t)^T x(t) = \sum_{i=0}^{d} x(t)_i \cdot x(t)_i > 0.$

- $w(t+1)^T x(t) = (w(t)+y(t)x(t))^T x(t) = w(t)^T x(t) + y(t)x(t)^T x(t)$

Same sign as y.

Perceptron Learning Algorithm (PLA)

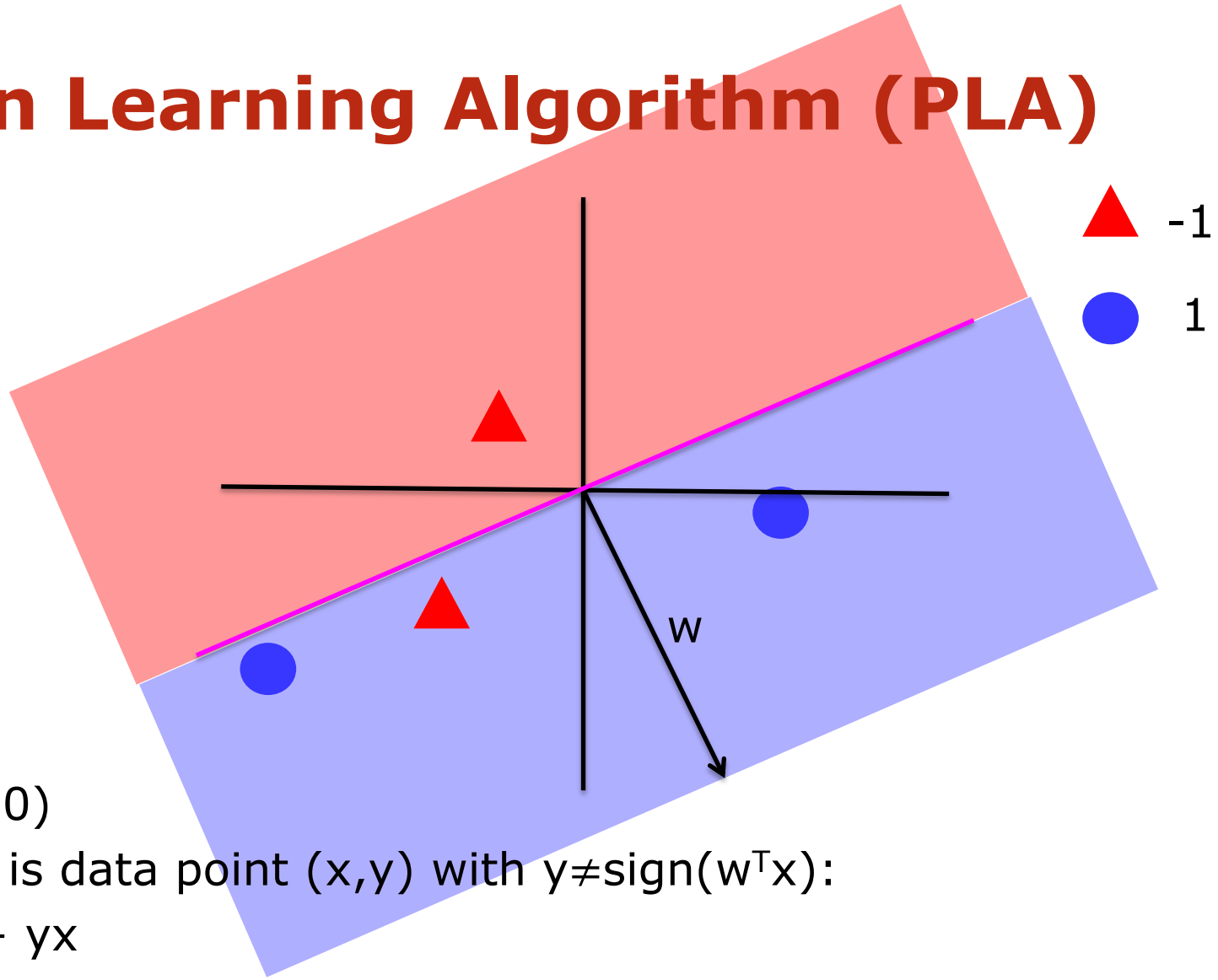
- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Make w "more" correct on (x,y)

- Observation:
  - May become wrong on other points!
  - Example…

Perceptron Learning Algorithm (PLA)

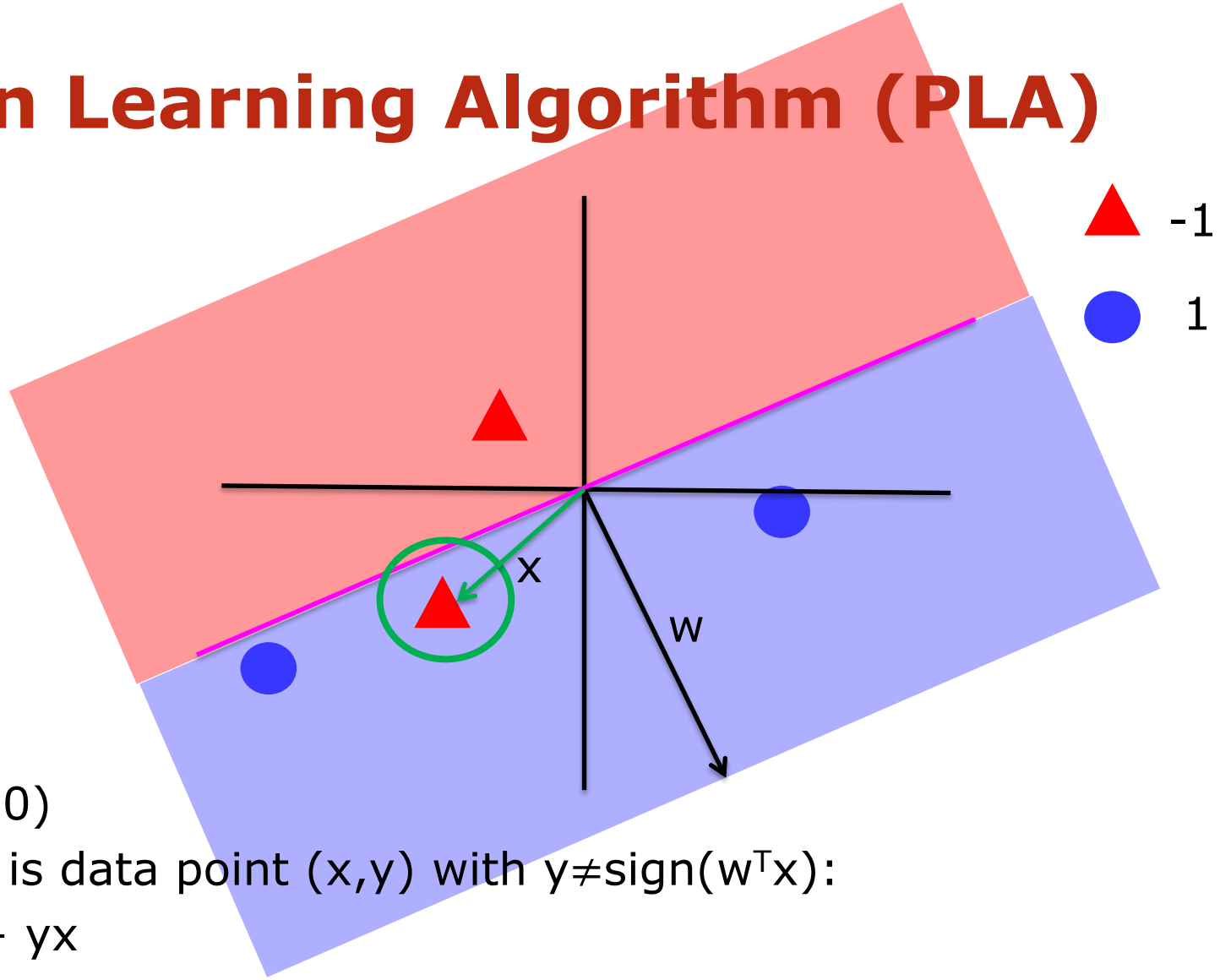
- Example:



- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Perceptron Learning Algorithm (PLA)

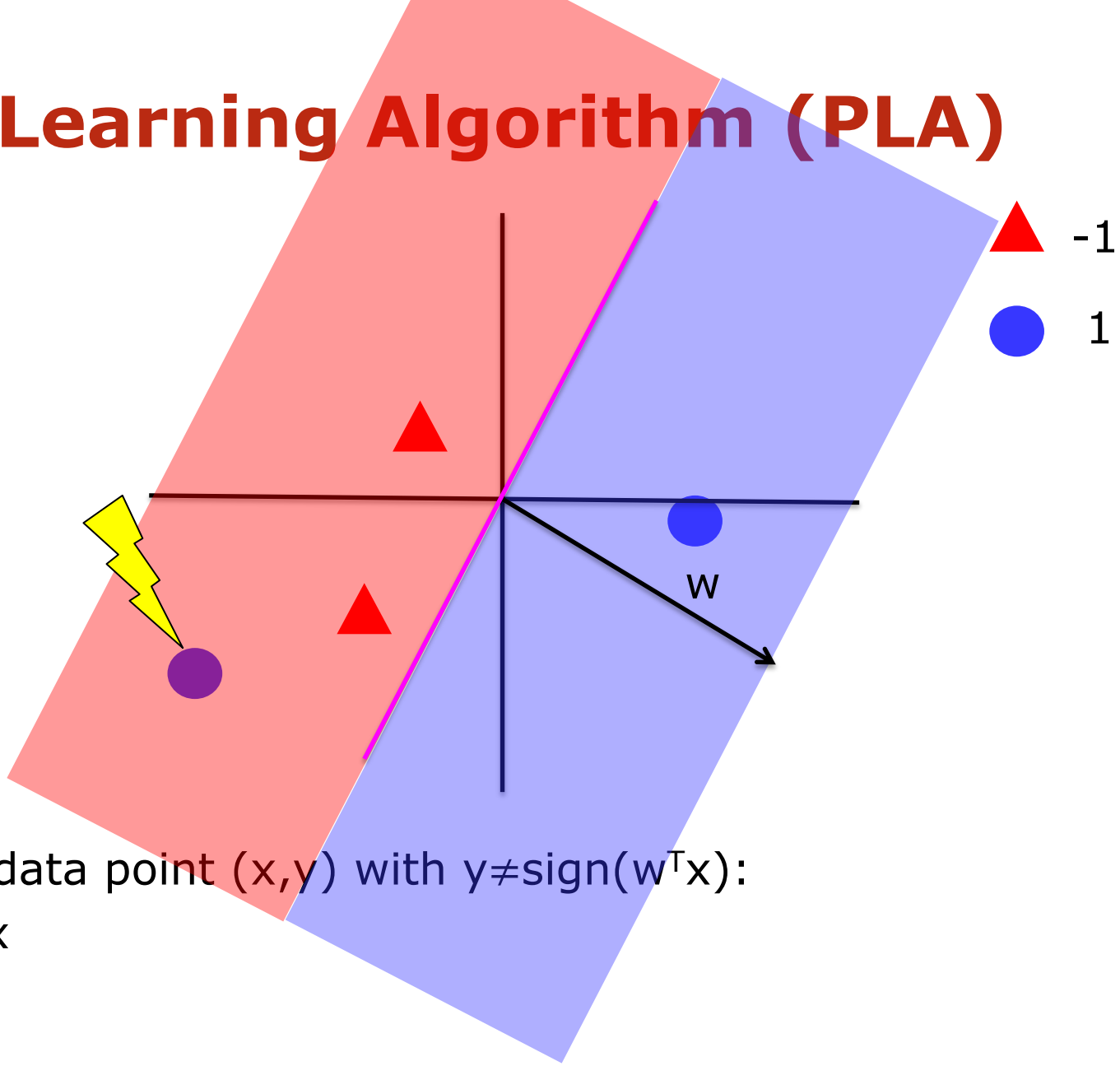
- Example:



- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Perceptron Learning Algorithm (PLA)

- Example:



- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Perceptron Learning Algorithm (PLA)

- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

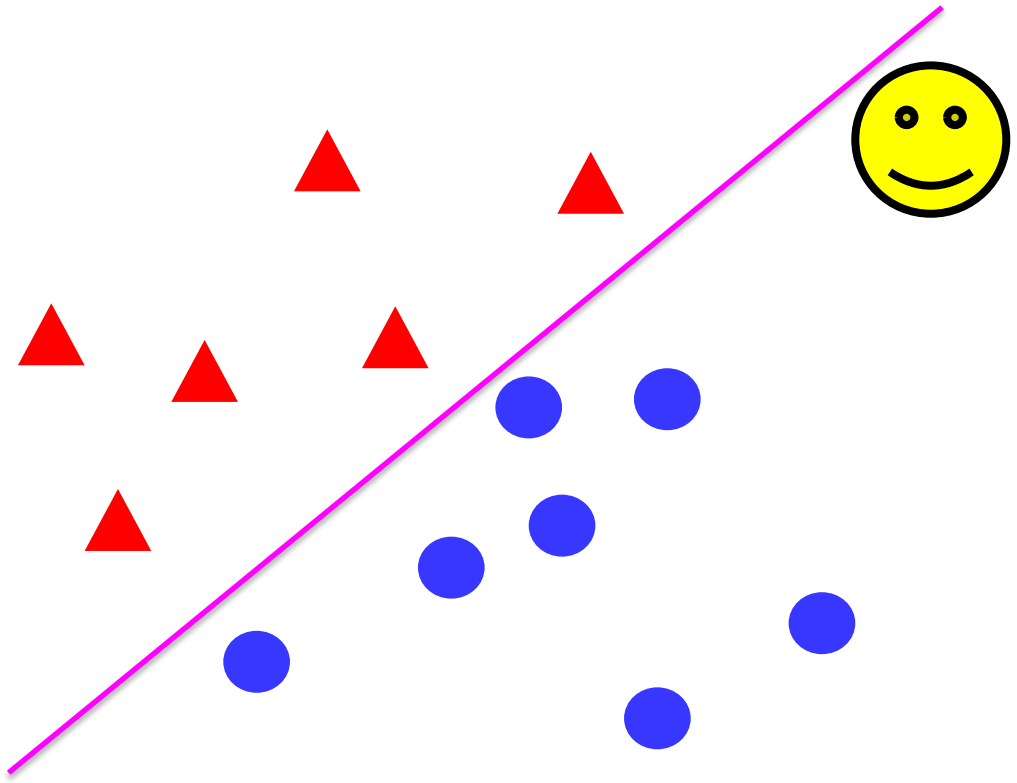
Make w "more" correct on (x,y)

- Observation:
  - May become wrong on other points!

- Is it even a good algorithm??
  - Can prove: If *linearly separable*, then terminates with perfect classification of training data.
  - Quite non-trivial to prove

Linearly Separable

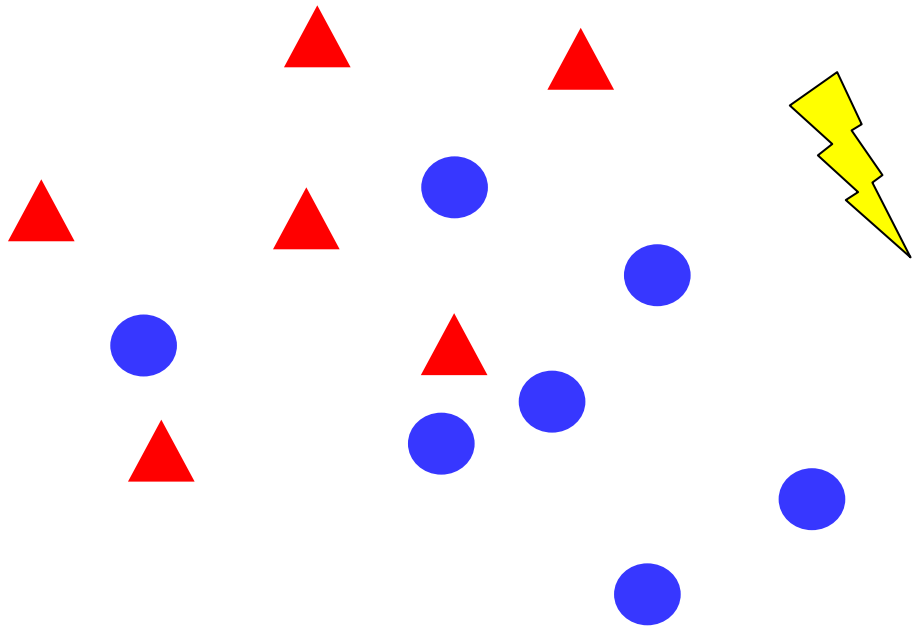
- Definition:
  - Linearly separable if there exists a separating hyperplane (line in 2d).



- Linearly separable!

Linearly Separable

- Definition:
  - Linearly separable if there exists a separating hyperplane (line in 2d).



- Not linearly separable!

Perceptron Learning Algorithm (PLA)

- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

- Demonstration:
  - Let's run some code and see it in action.

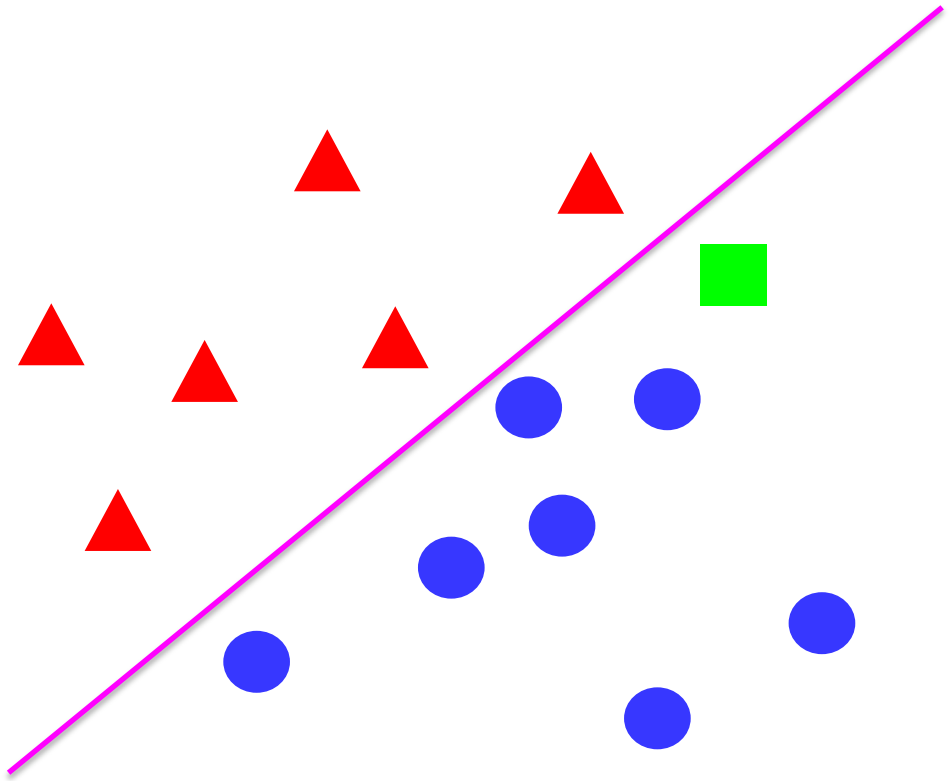
Perceptron Learning Algorithm (PLA)

- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

- Supervised Learning:
  - The returned w is the chosen algorithm/hypothesis.
  - Use that w to classify new data x:
    - Predict(x) = w(x) = $\text{sign}(w^T x)$.

Linearly Separable

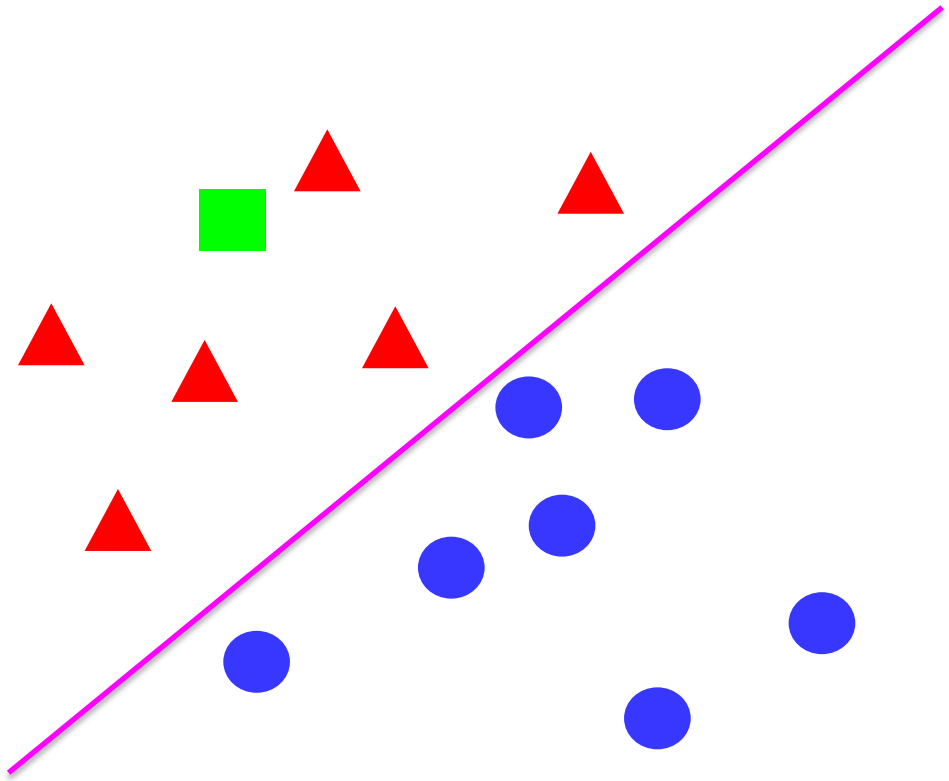
- Definition:
  - Linearly separable if there exists a separating hyperplane (line in 2d).



Predict(x) = sign($w^T x$) = blue

Linearly Separable

- Definition:
  - Linearly separable if there exists a separating hyperplane (line in 2d).



Predict(x) = sign($w^T x$) = red

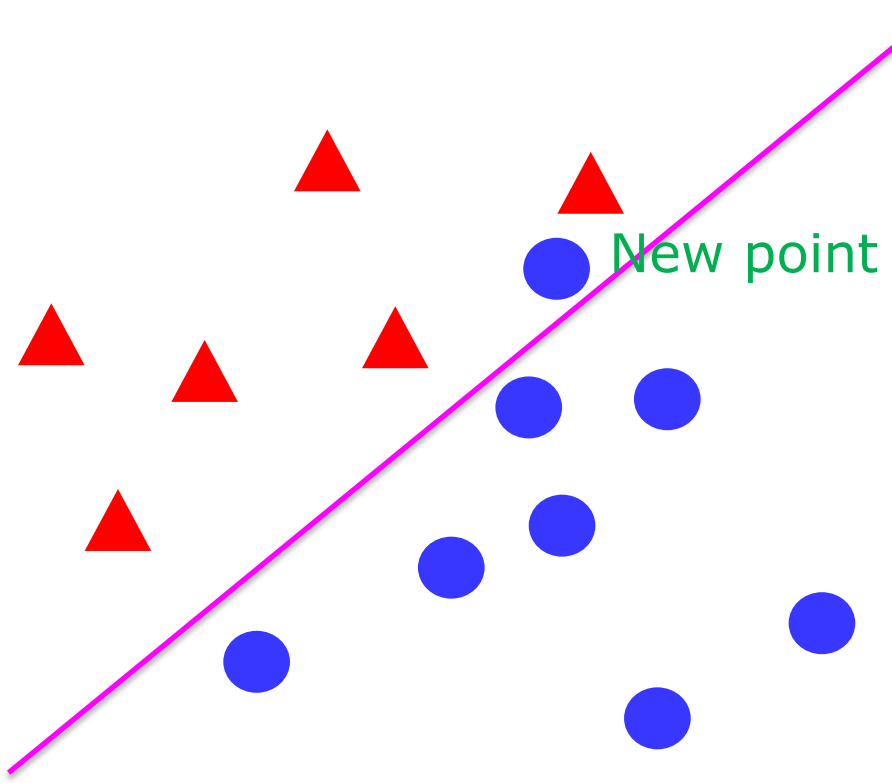
Perceptron Learning Algorithm (PLA)

- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

- Important:
  - Remember, perfect performance on training data does not imply perfect performance on new data!
    - But we hope it helps.
    - We'll soon return to this in-depth.

Linearly Separable

- Definition:
  - Linearly separable if there exists a separating hyperplane (line in 2d).



New data point could be very different!

Pocket PLA

- Would like to still do something useful if not linearly separable!
- Pocket version: Remember best w in your pocket.

- Algorithm:
  - w = (0,0,…,0)
  - bestScore = 0
  - bestW = w
  - While you have more time, let (x,y) have $y \neq \text{sign}(w^T x)$:
    - w = w + yx
    - score = percentage of data correctly classified by w.
    - If score>bestScore:
      - bestW=w, bestScore = score
  - Return bestW

Pocket PLA

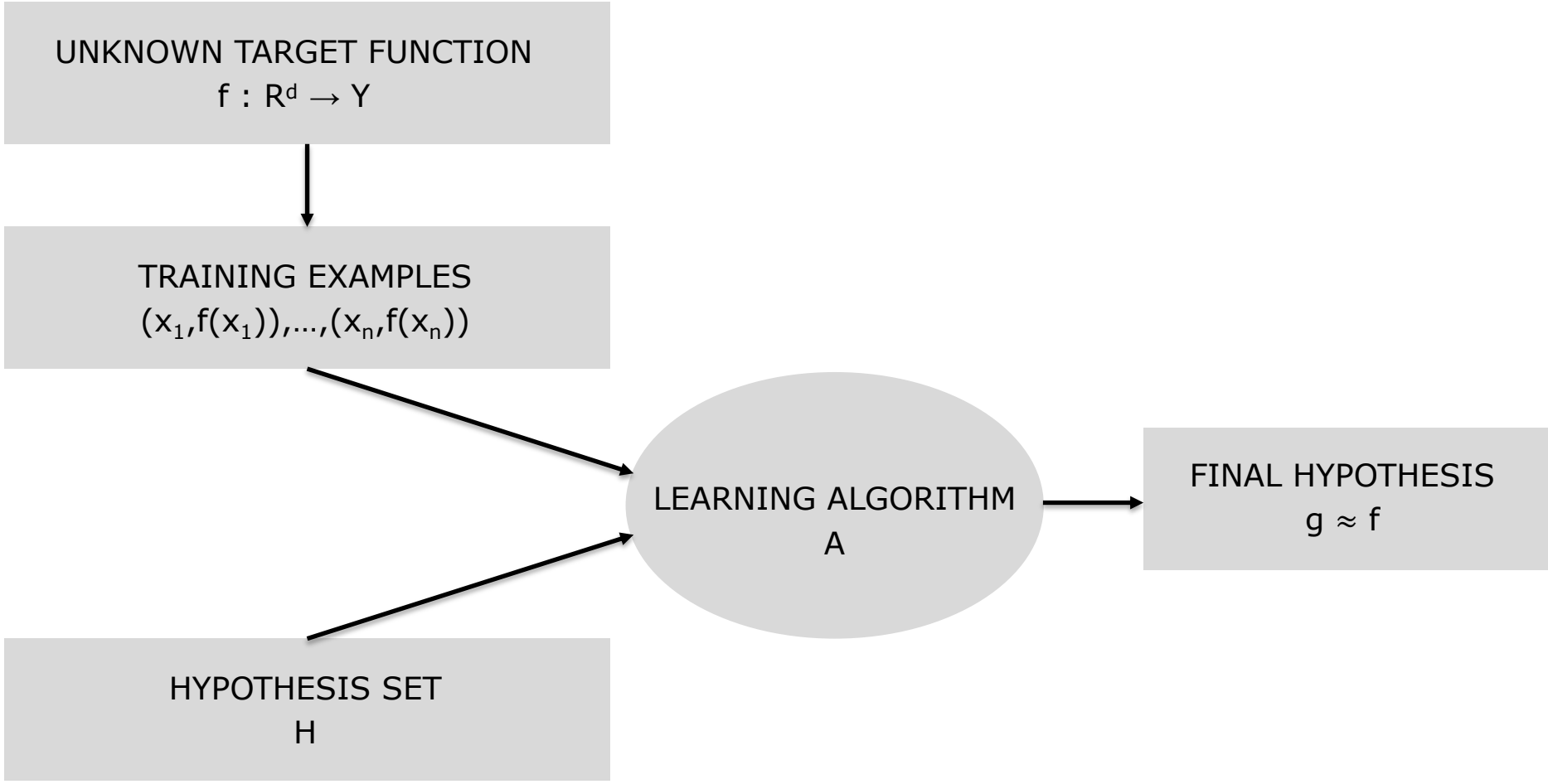
- Let's see it run!

- Algorithm:
  - w = (0,0,…,0)
  - bestScore = 0
  - bestW = w
  - While you have more time, let (x,y) have $y \neq \text{sign}(w^T x)$:
    - w = w + yx
    - score = percentage of data correctly classified by w.
    - If score>bestScore:
      - bestW=w, bestScore = score
  - Return bestW

Summary

- Motivated machine learning.
- Different variants:
  - Supervised
  - Unsupervised
  - Reinforcement
- In-depth with Supervised Learning.

Supervised Learning



Summary

- Motivated machine learning.
- Different variants: Supervised, Unsupervised, Reinforcement
- In-depth with Supervised Learning.

- Supervised Learning:
  - Try to get good performance on training data (Optimization).
    - Know labels
  - Hope it leads to good performance on new data (Generalization).
    - Don’t know labels
  - First example of learning algorithm:
    - Perceptron learning algorithm.
      - Perfect classification of training data if linearly separable.